

Tests for data-generating mechanisms

Stacy Cyrille

2026-03-11

Table of contents

1	Simulate binary variables	3
2	Simulate continuous variables	14
3	Simulate time-to-event data	29
3.1	Aalen additive model	29
3.2	Cox-Aalen model	48

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(survminer)
library(timereg)
library(gtsummary)

source(here::here("R", "utils.R"))
source(here::here("R", "generate_data.R"))
source(here::here("R", "performance_eval.R"))

options(scipen = 999)
```

1 Simulate binary variables

Example 1

We generate a simple example.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 3$
- Prevalence of DNA pathways: $\pi_1 = 0.1, \quad \pi_2 = 0.4, \quad \pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
set.seed(2026)

n <- 1000
prev <- c(0.1, 0.4, 0.6)

Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, 0.5,
                  0.2, 0.5, 1),
                3, 3)

dna1 <- data.frame(simulate_dna_pathways(n, prev, Sigma))
head(dna1)
```

```
  X1 X2 X3
1  0  1  1
2  0  0  0
3  0  1  1
4  0  0  0
5  0  0  0
6  0  0  0
```

```
# Check prevalences
colMeans(dna1)
```

```
  X1    X2    X3
0.093 0.405 0.603
```

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna1$X1, dna1$X2))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X2)
p-value = 1
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6397796 1.6015359
sample estimates:
odds ratio
 1.016585
```

```
fisher.test(table(dna1$X1, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X3)
p-value = 0.00005081
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.628539 4.962215
sample estimates:
odds ratio
 2.780409
```

```
fisher.test(table(dna1$X2, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X2, dna1$X3)
p-value < 0.00000000000000022
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 2.674120 4.802858
sample estimates:
```

odds ratio
3.573428

Example 2

We generate only uncorrelated variables.

- Sample size:

$$n = 1000$$

- Number of DNA pathways:

$$p = 3$$

- Prevalence of DNA pathways: $\pi_1 = 0.1$, $\pi_2 = 0.4$, $\pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
set.seed(2026)
Sigma0 <- diag(3)
dna0 <- data.frame(simulate_dna_pathways(n, prev, Sigma0))

# Check prevalences
colMeans(dna0)
```

```
      X1      X2      X3
0.111 0.409 0.591
```

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna0$X1, dna0$X2))
```

Fisher's Exact Test for Count Data

```
data:  table(dna0$X1, dna0$X2)
p-value = 0.3061
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
```

```
0.5128426 1.2159048
sample estimates:
odds ratio
0.7941041
```

```
fisher.test(table(dna0$X1, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X1, dna0$X3)
p-value = 0.8379
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6966913 1.6274484
sample estimates:
odds ratio
1.060629
```

```
fisher.test(table(dna0$X2, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X2, dna0$X3)
p-value = 0.3954
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6846429 1.1616203
sample estimates:
odds ratio
0.8916356
```

Example 3

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$

- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6, \quad \rho_{k2} = 0.5, \quad \rho_{k3} = 0.4, \quad \rho_{k4} = 0.1$
- Between-block correlation: $\rho_B = 0.05$

```
set.seed(2026)
```

```
p <- 20
```

```
# Generate random prevalences
```

```
prev <- runif(20, 0.1, 0.6)
```

```
prev
```

```
[1] 0.4493367 0.3782653 0.1700700 0.2428617 0.3776845 0.1125656 0.3331153
[8] 0.5305053 0.2262506 0.3904032 0.1029666 0.4459330 0.2155625 0.5242662
[15] 0.1769176 0.2784047 0.3726023 0.1005988 0.2589122 0.1086447
```

```
# Initialize the matrix
```

```
Sigma <- generate_block_corr(p,
```

```
    block_size = 5,
```

```
    rho_within = c(0.6, 0.5, 0.4, 0.1),
```

```
    rho_between = 0.05)
```

```
# Inspect the matrix
```

```
colnames(Sigma) <- paste0("X", 1:p)
```

```
rownames(Sigma) <- paste0("X", 1:p)
```

```
Sigma %>%
```

```
  as.data.frame() %>%
```

```
  tibble::rownames_to_column("var1") %>%
```

```
  pivot_longer(-var1, names_to = "var2") %>%
```

```
  mutate(value = ifelse(var1 == var2, NA, value),
```

```
         var1 = factor(var1, levels = paste0("X", 1:p)),
```

```
         var2 = factor(var2, levels = paste0("X", 1:p))) %>%
```

```
{
```

```
  ggplot(., aes(var1, var2, fill = value)) +
```

```
    geom_tile() +
```

```
    scale_fill_distiller(palette = "RdBu", direction = -1,
```

```
                        limits = c(-1, 1)) +
```

```
    theme_minimal() +
```

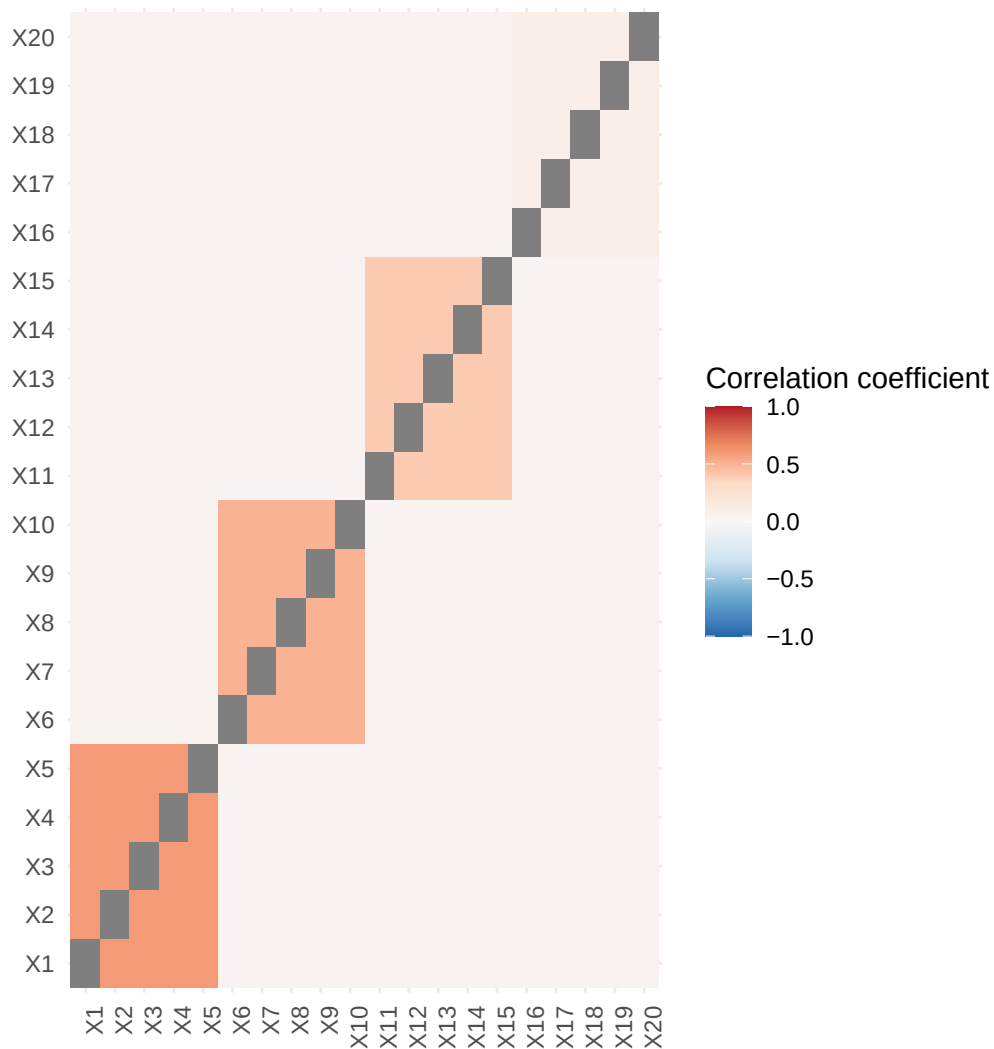
```
    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
```

```
    labs(fill = "Correlation coefficient", x = "", y = "",
```

```
         title = "True correlation matrix in the simulation design")
```

```
}
```

True correlation matrix in the simulation design



```
# Simulate correlated data using this matrix
dna_block <- data.frame(simulate_dna_pathways(n, prev, Sigma))

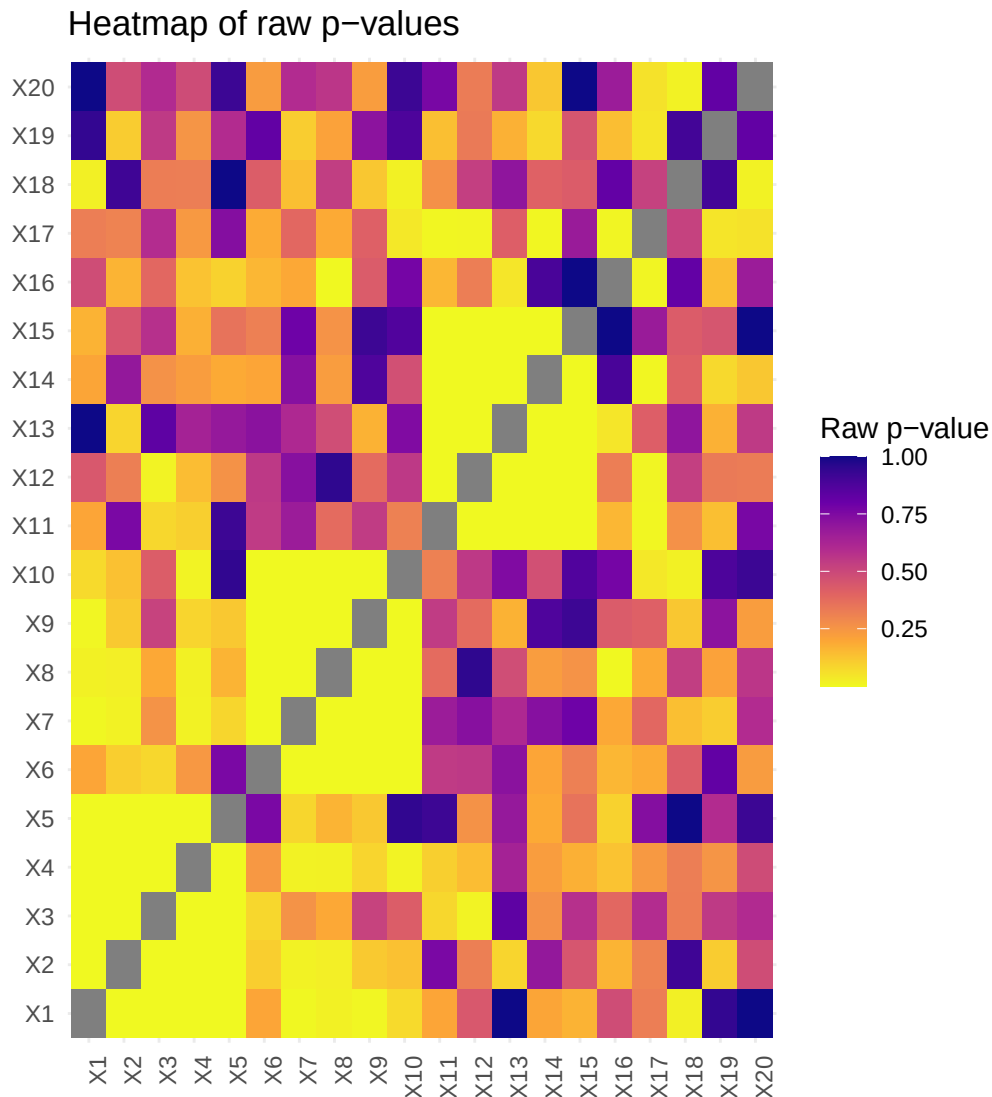
# Check marginal prevalences
data.frame(true_prev = prev,
            est_prev = colMeans(dna_block))%>%
  rownames_to_column("variable")%>%
  kable(format = "html", caption = "True and empirical prevalences",
        col.names = c("Variable", "True prevalence", "Estimated prevalence"))
```


Table 1: True and empirical prevalences

Variable	True prevalence	Estimated prevalence
X1	0.4493367	0.463
X2	0.3782653	0.367
X3	0.1700700	0.161
X4	0.2428617	0.226
X5	0.3776845	0.371
X6	0.1125656	0.119
X7	0.3331153	0.298
X8	0.5305053	0.523
X9	0.2262506	0.199
X10	0.3904032	0.376
X11	0.1029666	0.119
X12	0.4459330	0.434
X13	0.2155625	0.207
X14	0.5242662	0.522
X15	0.1769176	0.189
X16	0.2784047	0.287
X17	0.3726023	0.372
X18	0.1005988	0.105
X19	0.2589122	0.247
X20	0.1086447	0.122

```
# 20x20 matrix of raw p-values
pval_block <- pairwise_fisher(dna_block, names(dna_block))

# Heatmap of raw p-values
mat_long <- as.data.frame(as.table(pval_block$p.value.matrix))%>%
  mutate(Freq = ifelse(Var1==Var2, NA, Freq))
ggplot(mat_long, aes(Var1, Var2, fill = Freq)) +
  geom_tile() +
  scale_fill_viridis_c(option = "C", direction = -1) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  labs(fill = "Raw p-value", x = "", y = "",
       title = "Heatmap of raw p-values")
```



All simulations were repeated across $R = 100$ replications.

For each replicate, pairwise associations between DNA-based pathways were tested using Fisher's exact test, and the empirical type I error was estimated as the proportion of replications with p-values below the nominal significance level ($\alpha = 0.05$)

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$
- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6, \quad \rho_{k2} = 0.5, \quad \rho_{k3} = 0.4, \quad \rho_{k4} = 0.05$
- Between-block correlation: $\rho_B = 0.01$

```

set.seed(2026)
R <- 1000
n <- 1000
p <- 20
prev <- runif(p, 0.1, 0.6) %>%
  set_names(paste0("X",1:p))
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.6, 0.5, 0.4, 0.05),
                             rho_between = 0.01)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("D",1:20)),
         var2 = factor(var2, levels = paste0("D",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                           limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }

# Run simulations
dna_sims <- replicate(R,
                      simulate_dna_pathways(n, prev, Sigma),

```

```

        simplify = FALSE) %>%
imap(~ mutate(data.frame(.x), R = .y)) %>%
bind_rows()

# Prevalence bias
dna_sims %>%
  bind_rows()%>%
  pivot_longer(-R, names_to = "pathway", values_to = "est_prev") %>%
  dplyr::full_join(rownames_to_column(data.frame(true_prev = prev),
        "pathway"), by = "pathway")%>%

  group_by(pathway) %>%
  summarize(bias = estimate_bias(true = true_prev, est = est_prev),
        pathway = factor(pathway, levels = paste0("D", 1:20)),
        .groups = "drop")%>%
  {
    ggplot(., aes(x = pathway, y = bias, colour = pathway))+
      geom_point(aes(fill = "pathway"))+
      geom_hline(yintercept = 0, linetype = "dashed")+
      scale_y_continuous(limits = c(-0.05, 0.05))+
      labs(x = "Pathway", y = "Bias", title = "Prevalence bias")+
      theme_minimal()+
      theme(legend.position = "none")
  }

# Type I error
dna_sims_pval <- dna_sims %>%
  group_split(R) %>%
  map(~.x %>%
    dplyr::select(-R)%>%
    pairwise_fisher(vars = names()))

# Heatmap of mean raw p-values
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
    mutate(Freq = ifelse(Var1==Var2, NA, Freq))%>%
    reduce(bind_rows, .id = "R")%>%
    group_by(Var1, Var2)%>%
    summarise(mean_p.value = mean(Freq), .groups = "drop")%>%
  {
    ggplot(., aes(Var1, Var2, fill = mean_p.value)) +
      geom_tile() +
      scale_fill_viridis_c(option = "C", direction = -1) +

```

```

    theme_minimal() +
    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
    labs(fill = "Mean raw p-value", x = "", y = "",
         title = "Heatmap of mean raw p-values")
  }

# Heatmap of the proportion of replications with p-values < 0.05
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
      mutate(Freq = ifelse(Var1==Var2, NA, Freq)))%>%
  reduce(bind_rows, .id = "R")%>%
  group_by(Var1, Var2)%>%
  summarise(mean_typeI = mean(Freq < 0.05, na.rm = TRUE),
            .groups = "drop")%>%
  {
    ggplot(., aes(Var1, Var2, fill = mean_typeI)) +
      geom_tile() +
      scale_fill_viridis_c(option = "C", direction = -1) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Proportion of replications \nwith p-values < 0.05", x = "", y = "",
           title = "Heatmap of the proportion of replications with p-values < 0.05")
  }

```

2 Simulate continuous variables

Example 1

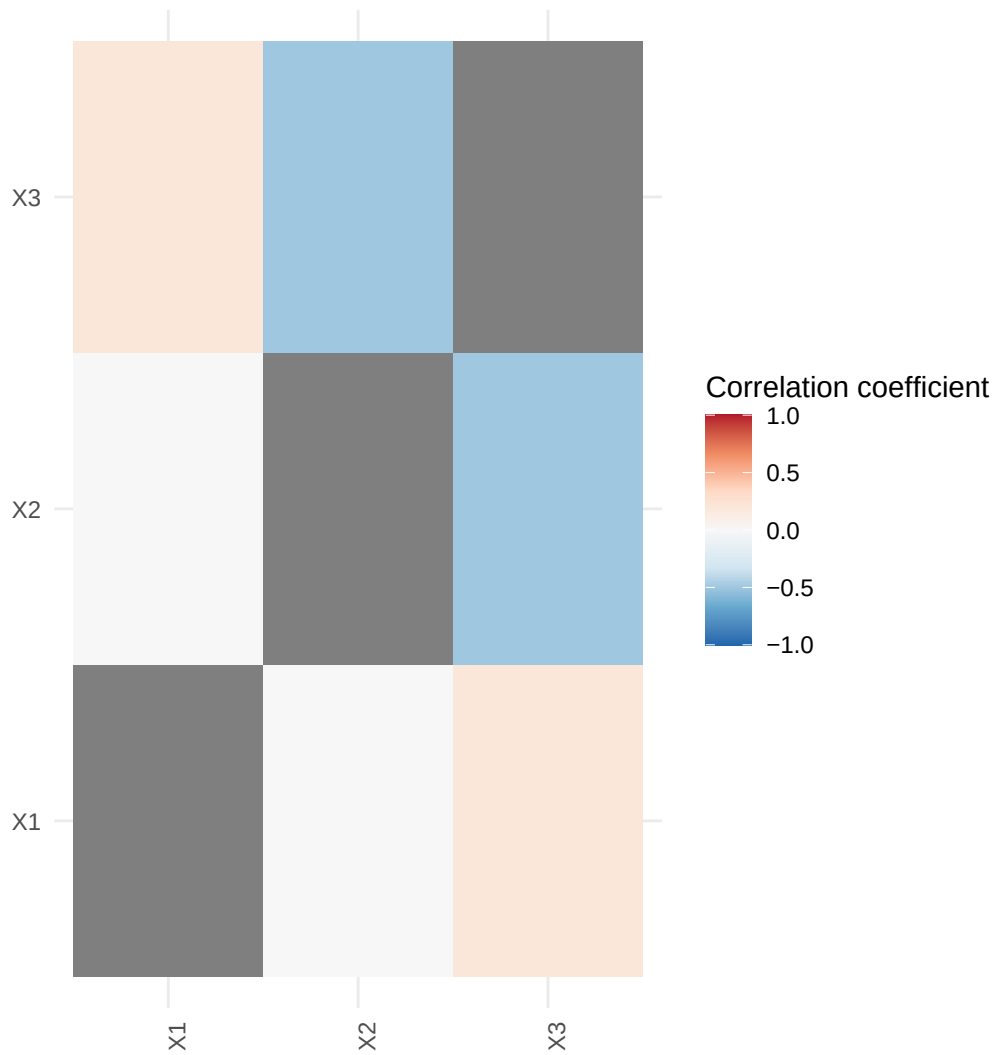
We generate a simple example.

- Sample size: $n = 1000$
- Number of RNA pathways: $p = 3$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
set.seed(2026)
p <- 3
mu <- 0
Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, -0.5,
                  0.2, -0.5, 1),
                3, 3)
# Inspect the matrix
colnames(Sigma) <- paste0("X",1:p)
rownames(Sigma) <- paste0("X",1:p)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:p)),
         var2 = factor(var2, levels = paste0("X",1:p)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation design



```
rna1 <- simulate_rna_pathways(n, p, mu, Sigma)
```

```
# Check mean
```

```
colMeans(rna1)
```

```

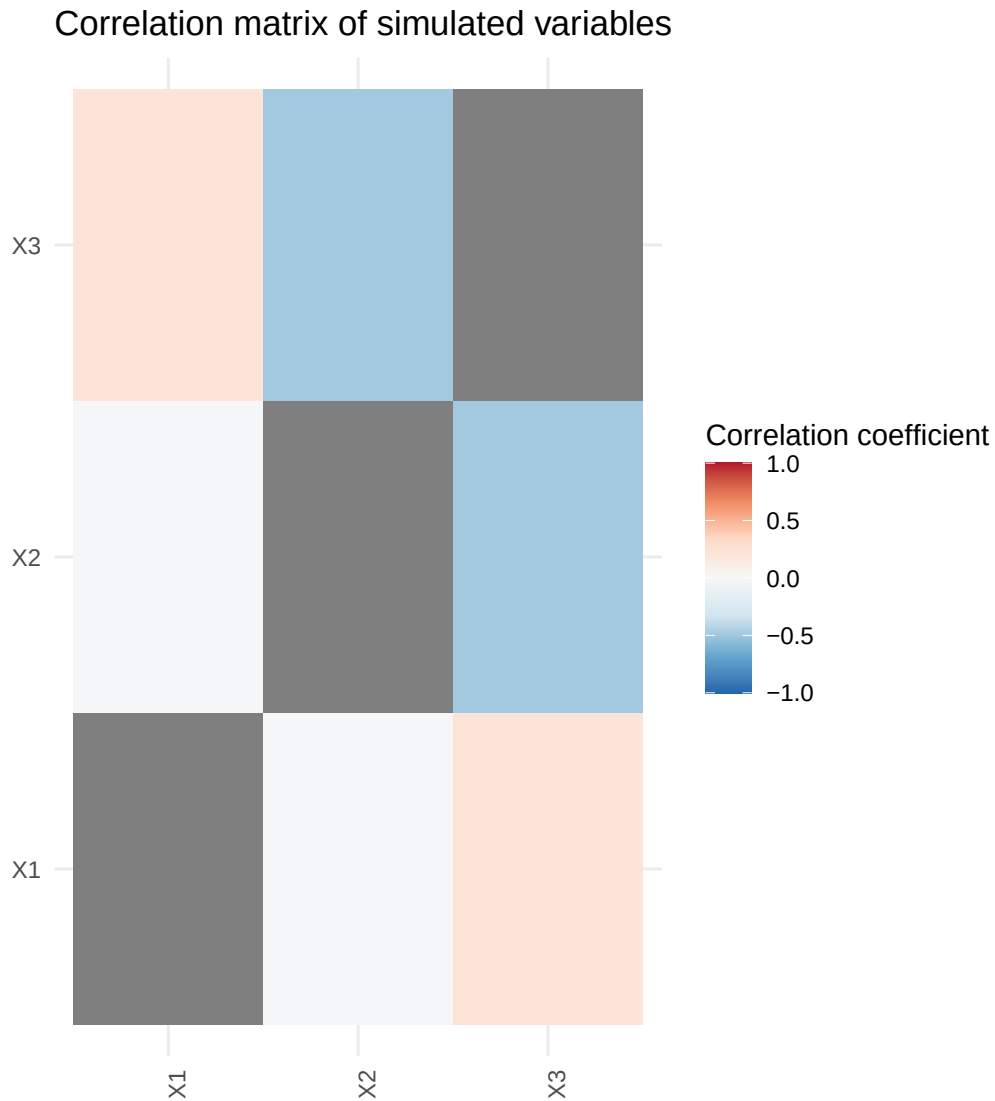
              X1              X2
-0.0000000000000000206328010 -0.000000000000000003625572
              X3
-0.00000000000000000156203175

```

```
# Check variance
apply(rna1, 2, var)
```

```
X1 X2 X3
1 1 1
```

```
# Correlation matrix
as.data.frame(cor(rna1))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Correlation matrix of simulated variables")
  }
```

Example 2

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$
- Block size: $b = 5$
- Number of blocks: $K = 4$

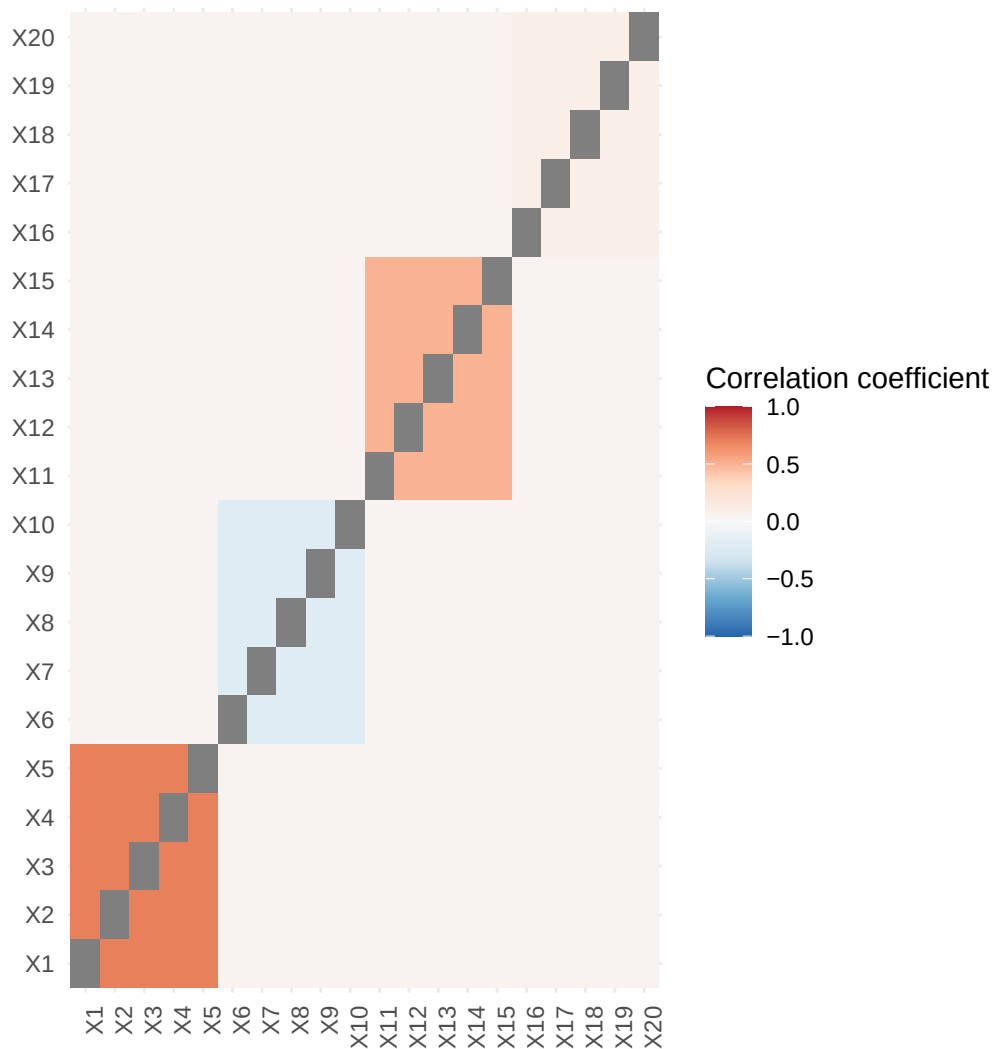
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.1$
- Between-block correlation: $\rho_B = 0.05$

```
set.seed(2026)
p = 20

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.1),
                             rho_between = 0.05)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                           limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation design



```
# Simulate correlated data using this matrix
rna_block <- simulate_rna_pathways(n, p, mu, Sigma)

# Check mean
colMeans(rna_block)
```

```

              X1              X2
0.000000000000000014829500 0.0000000000000000020793697
              X3              X4
```

```

0.000000000000000016604773 -0.000000000000000028857992
                        X5                        X6
0.000000000000000024050206 0.000000000000000013582885
                        X7                        X8
0.00000000000000002560452 -0.000000000000000015402610
                        X9                        X10
-0.000000000000000025066754 0.00000000000000005332540
                        X11                       X12
0.000000000000000032315296 0.00000000000000001650589
                        X13                       X14
0.000000000000000019556405 -0.00000000000000002100750
                        X15                       X16
-0.000000000000000016658550 -0.000000000000000012268832
                        X17                       X18
-0.000000000000000014805540 -0.000000000000000019619723
                        X19                       X20
-0.000000000000000004916206 -0.00000000000000004945697

```

```

# Check variance
apply(rna_block, 2, var)

```

```

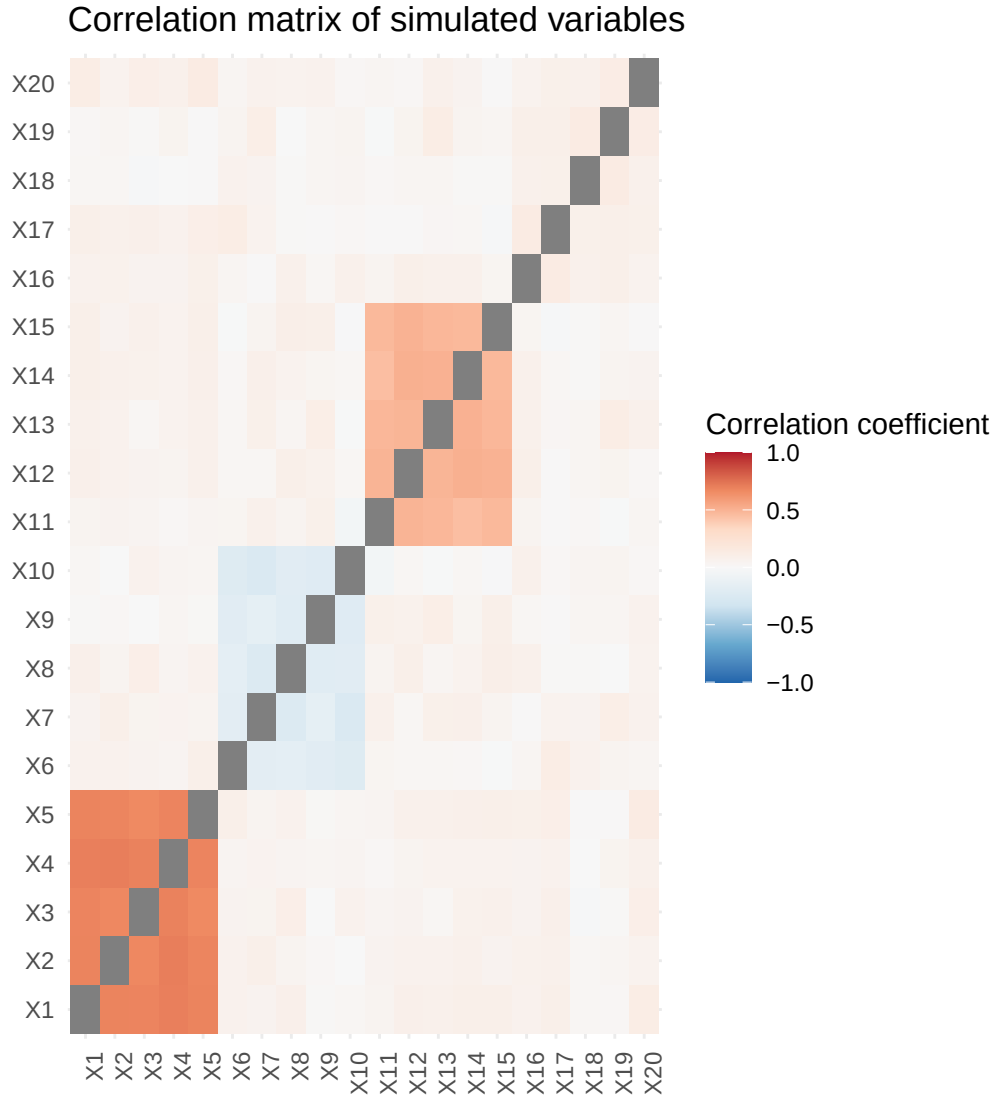
X1  X2  X3  X4  X5  X6  X7  X8  X9  X10  X11  X12  X13  X14  X15  X16  X17  X18  X19  X20
1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1

```

```

# Correlation matrix
as.data.frame(cor(rna_block))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Correlation matrix of simulated variables")
  }

```



All simulations were repeated across $R = 100$ replications.

For each pathway, we computed the empirical mean, variance, and pairwise correlation coefficients of the RNA pathways across simulation replicates. Mean, variance and correlation coefficients were averaged across replicates.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$

- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.3$
- Between-block correlation: $\rho_B = 0.1$

```

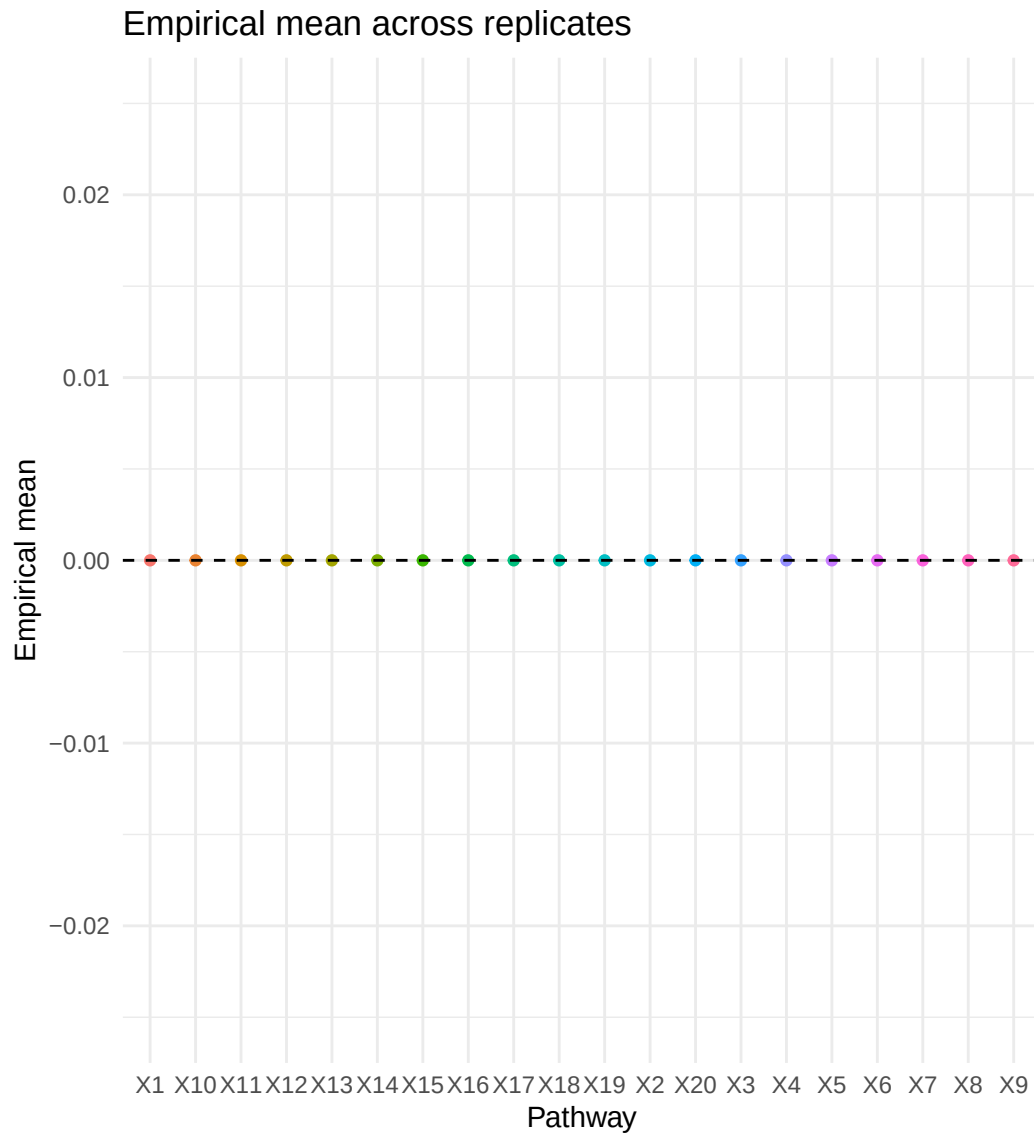
set.seed(2026)
R <- 1000
p <- 20
mu <- 0

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.3),
                             rho_between = 0.1)
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)

# Run simulations
rna_sims <- replicate(R,
                      simulate_rna_pathways(n, p, mu, Sigma),
                      simplify = FALSE)

# Check mean
rna_sims %>%
  map(~colMeans(.x))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 0, linetype = "dashed")+
      scale_y_continuous(limits = c(-0.025, 0.025))+
      labs(x = "Pathway", y = "Empirical mean",
           title = "Empirical mean across replicates")+
      theme_minimal()+
      theme(legend.position = "none")
  }

```



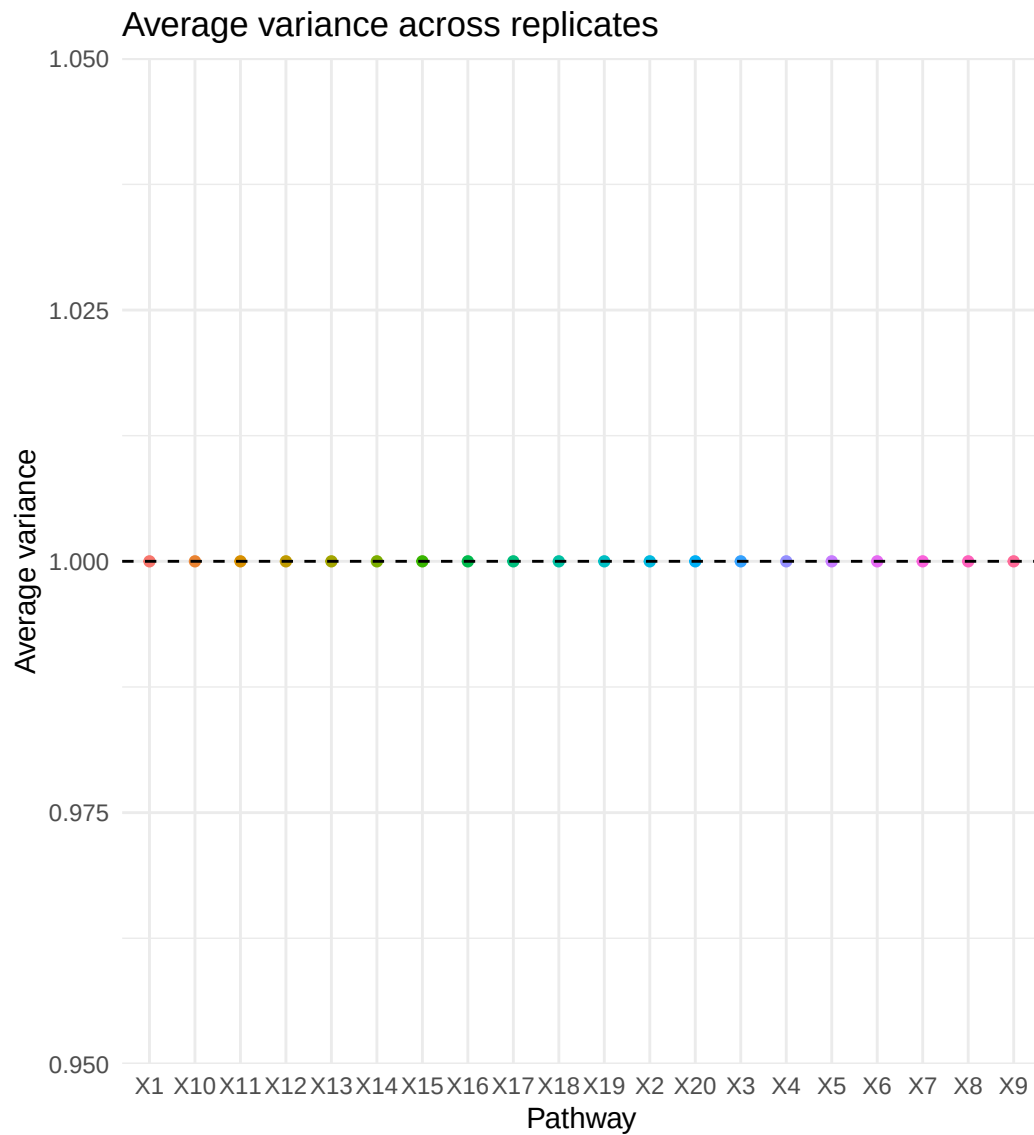
```
# Check variance
rna_sims %>%
  map(~ apply(., 2, var))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 1, linetype = "dashed")+

```

```

labs(x = "Pathway", y = "Average variance",
      title = "Average variance across replicates")+
theme_minimal()+
theme(legend.position = "none")
}

```



```

# True correlation matrix
Sigma %>%
  as.data.frame()%>%

```

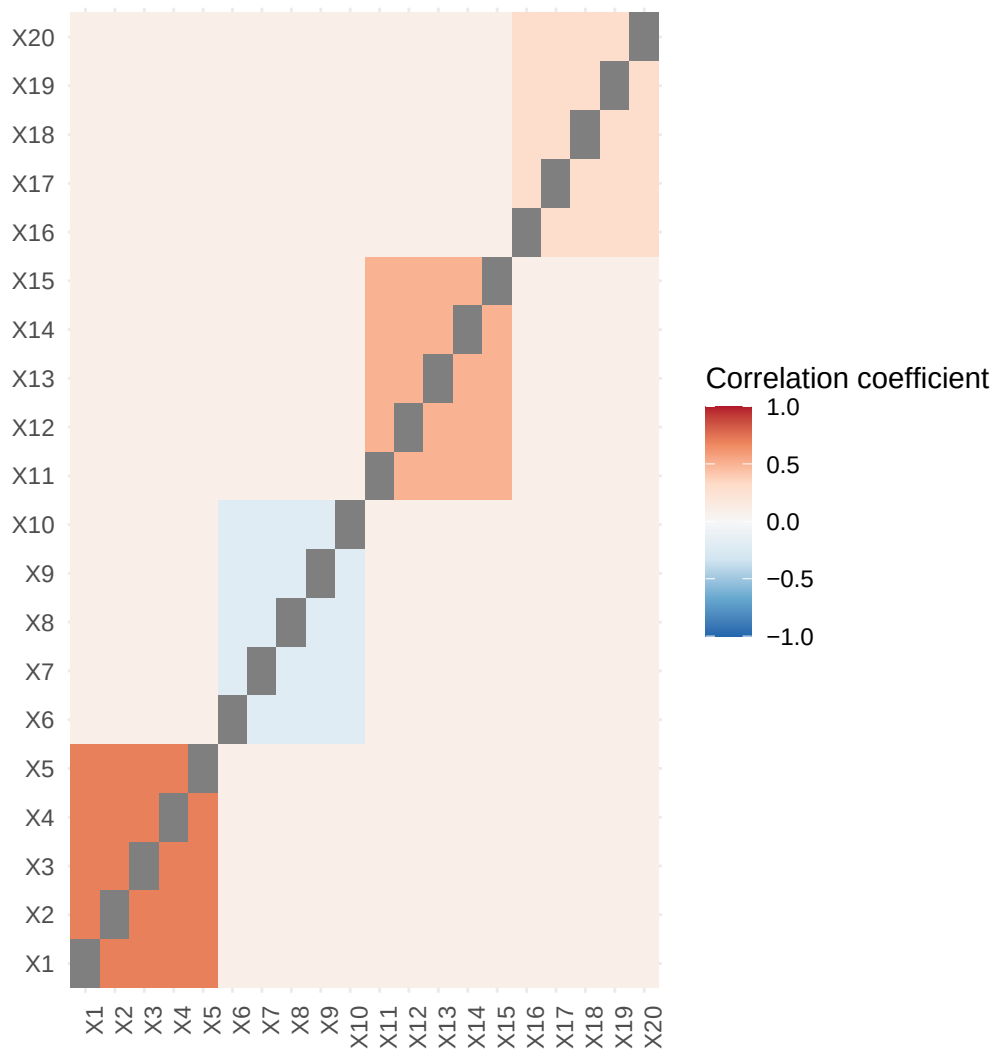


```

tibble::rownames_to_column("var1")%>%
pivot_longer(-var1, names_to = "var2")%>%
mutate(value = ifelse(var1==var2, NA, value),
       var1 = factor(var1, levels = paste0("X",1:20)),
       var2 = factor(var2, levels = paste0("X",1:20)))%>%
{
  ggplot(., aes(var1, var2, fill = value)) +
    geom_tile() +
    scale_fill_distiller(palette = "RdBu", direction = -1,
                        limits = c(-1,1)) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
    labs(fill = "Correlation coefficient", x = "", y = "",
         title = "True correlation matrix in the simulation design")
}

```

True correlation matrix in the simulation design



```
# Average correlation matrix across simulation replicates
cor_sims_rna <- rna_sims %>%
  map(cor) %>%
  reduce(`+`) / length(rna_sims)

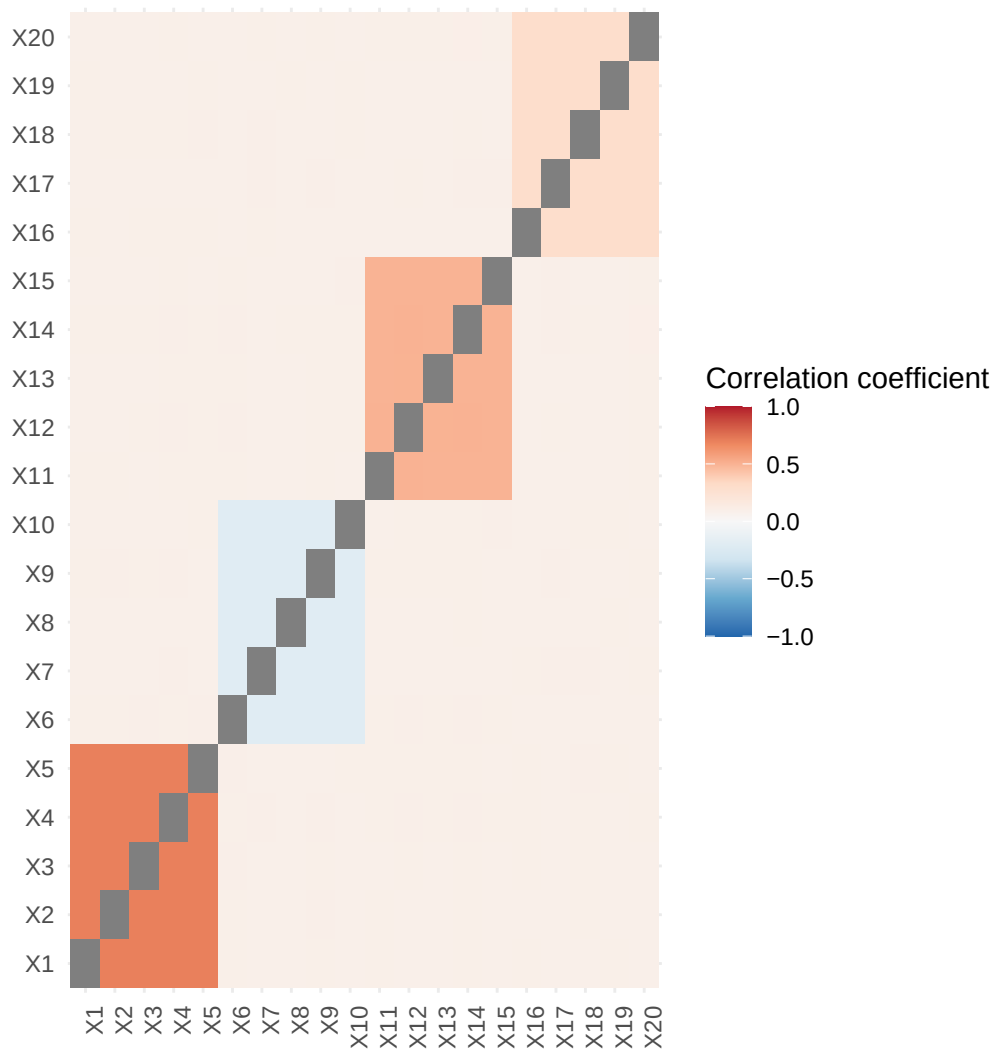
as.data.frame(cor_sims_rna)%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
```

```

    var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                           limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Average correlation matrix across simulation replicates")
  }

```

Average correlation matrix across simulation replicates



3 Simulate time-to-event data

3.1 Aalen additive model

Four binary covariates were generated *independently* with prevalences spanning rare to frequent alterations, $X_j \sim \text{Bernoulli}(\pi_j)$ with $\pi_j = (0.005, 0.02, 0.05, 0.20)$.

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$$

with $\lambda_0 = 0.05$ and $\beta = (0.06, 0.02, 0.005, 0)$.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 5%, and the calibrated value was then fixed across all simulation replicates.

```
# Set seed for reproducibility
set.seed(2026)

# Generate data
n <- 1000
p <- 4
sigma <- diag(x = 1, nrow = p, ncol = p)
prev <- c(0.005, 0.02, 0.05, 0.20)
beta <- c(0.06, 0.02, 0.005, 0)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                               beta         = beta,
                               prev         = prev,
                               sigma        = sigma,
                               target_cens  = target_cens,
                               seed         = 2026)

c_max
```

```
[1] 395.9839
```

```
data <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
head(data)
```

	time	event	X1	X2	X3	X4
1	13.637923	1	0	0	0	1
2	5.971854	1	0	0	0	1
3	11.125716	1	0	0	0	0
4	5.967318	1	0	0	0	0
5	8.175295	1	0	0	0	0
6	23.579391	1	0	0	0	0

```
# Check prevalences
colMeans(data %>% dplyr::select(starts_with("X")))
```

	X1	X2	X3	X4
	0.007	0.020	0.052	0.201

```
# Check censoring rate
mean(1 - data$event)
```

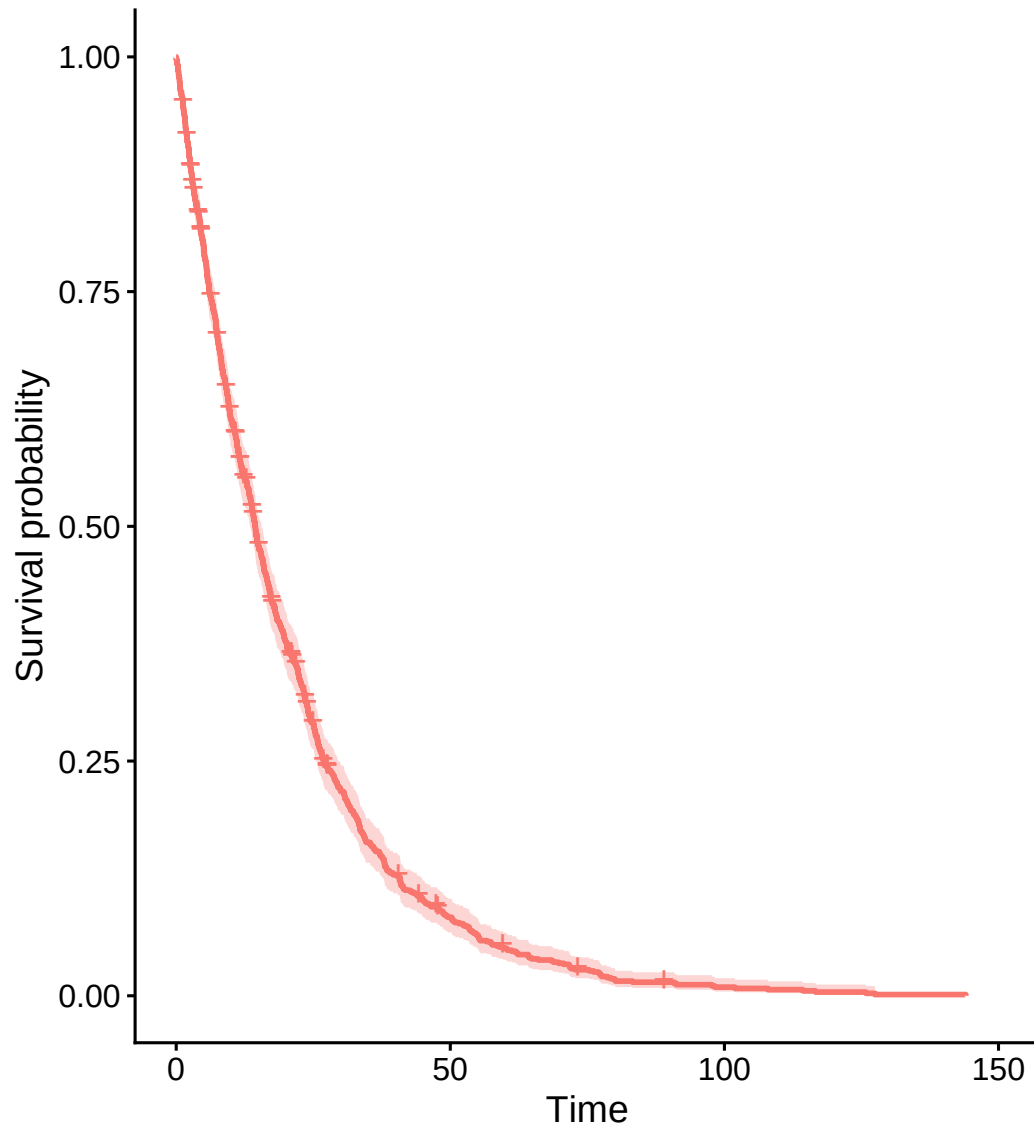
```
[1] 0.043
```

```
# Kaplan-Meier
km_aalen_fit <- survfit(Surv(time, event)~1, data)
km_aalen_fit
```

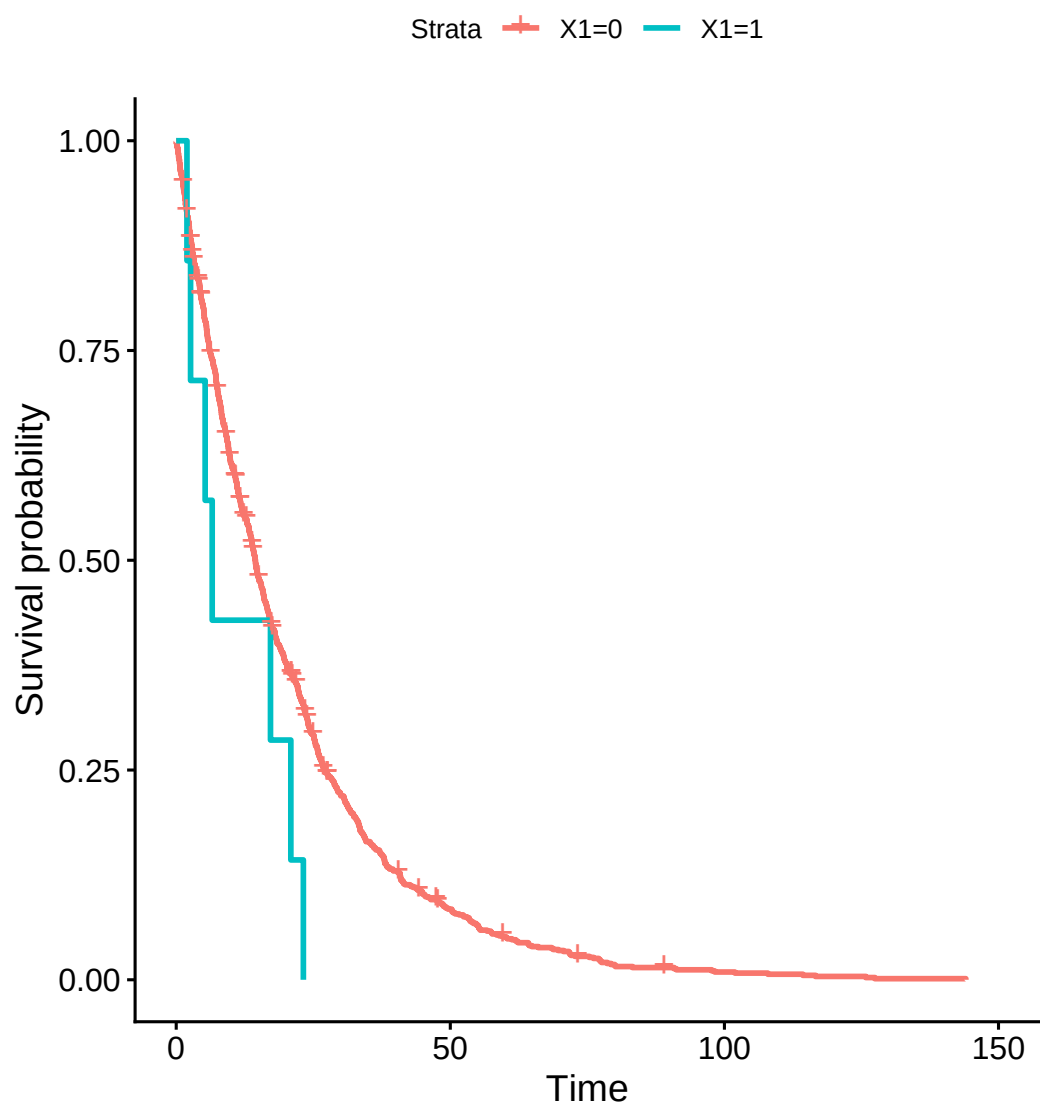
```
Call: survfit(formula = Surv(time, event) ~ 1, data = data)
```

	n	events	median	0.95LCL	0.95UCL
[1,]	1000	957	14.4	13.5	15.6

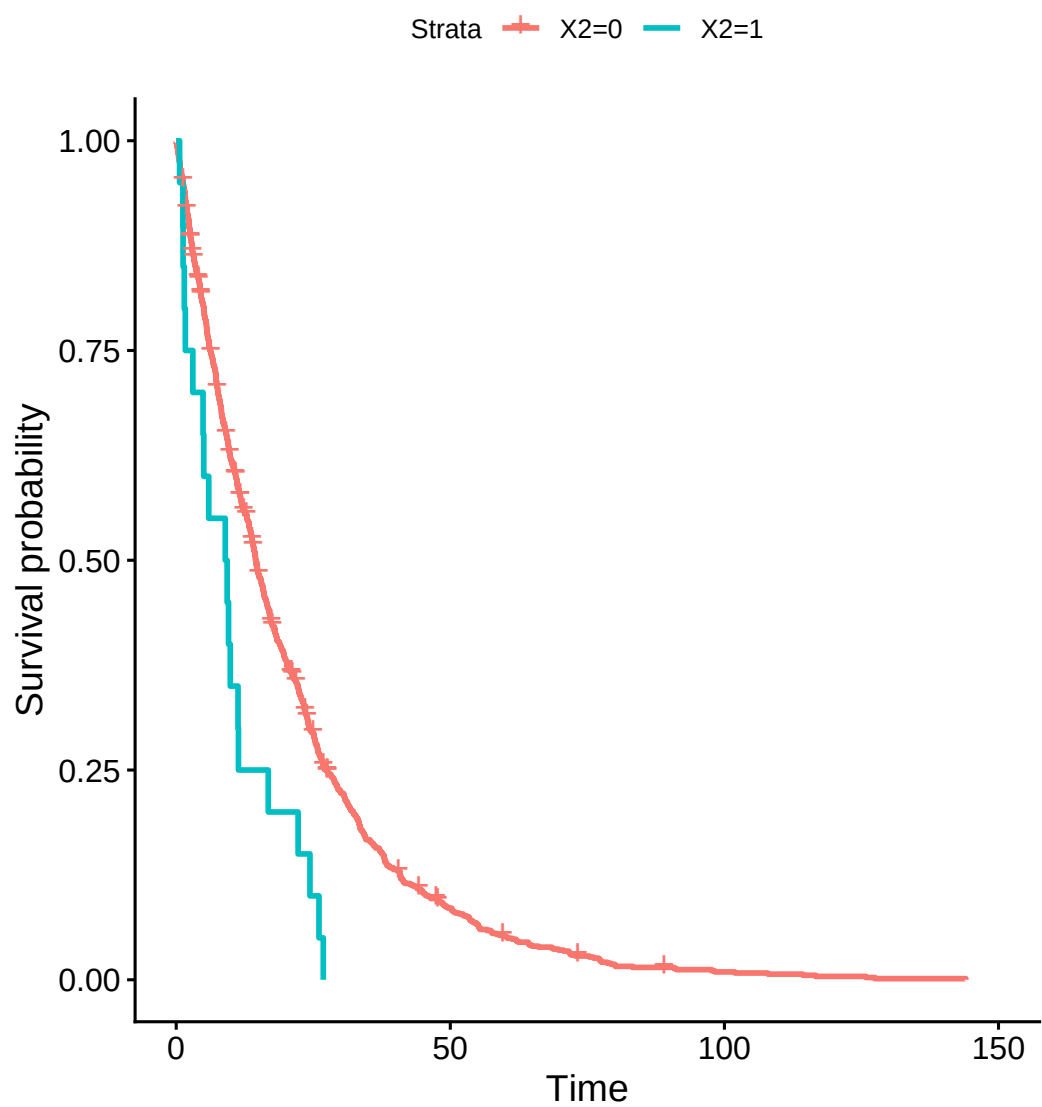
```
ggsurvplot(km_aalen_fit, data = data, legend = "none")
```



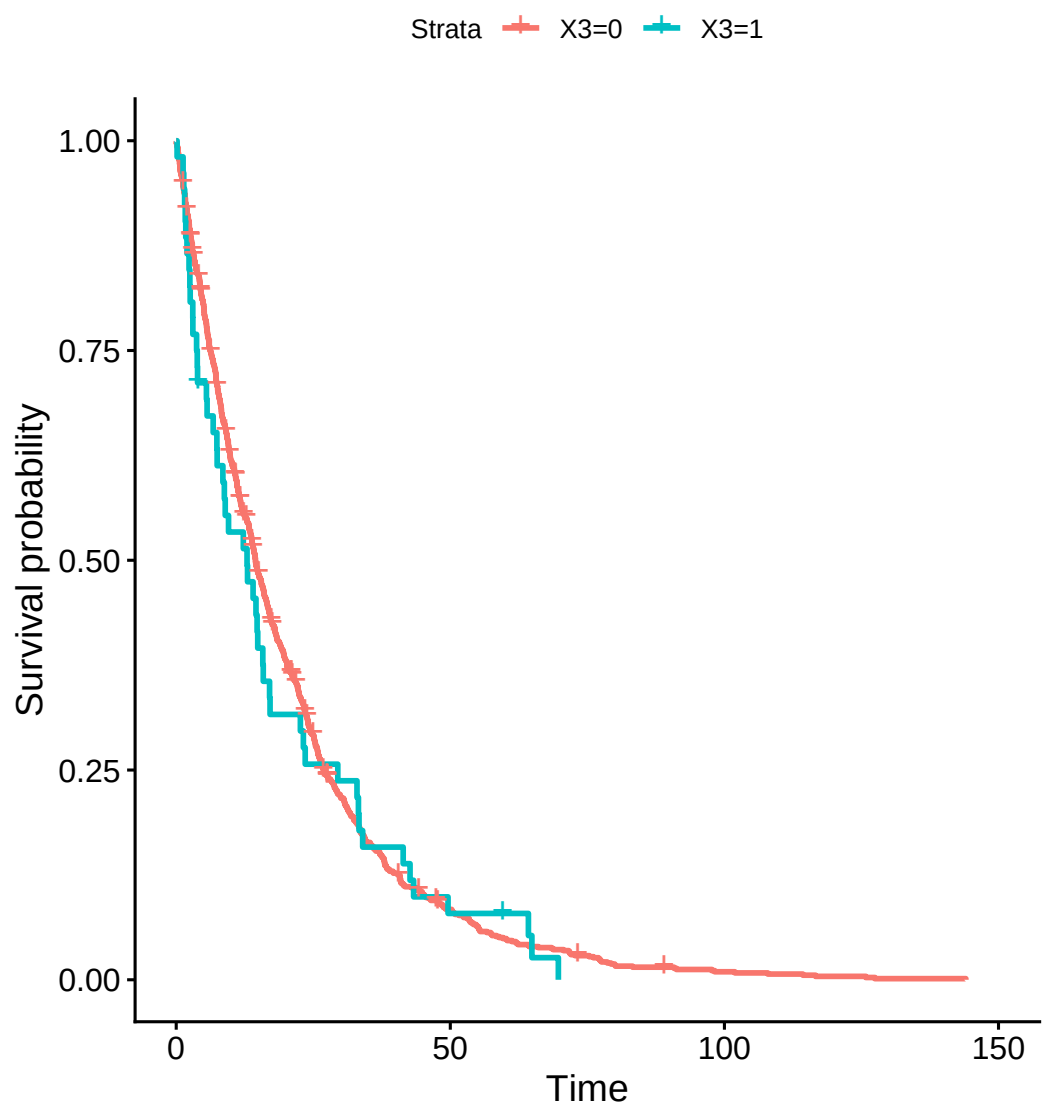
```
ggsurvplot(survfit(Surv(time, event)~X1, data), data)
```



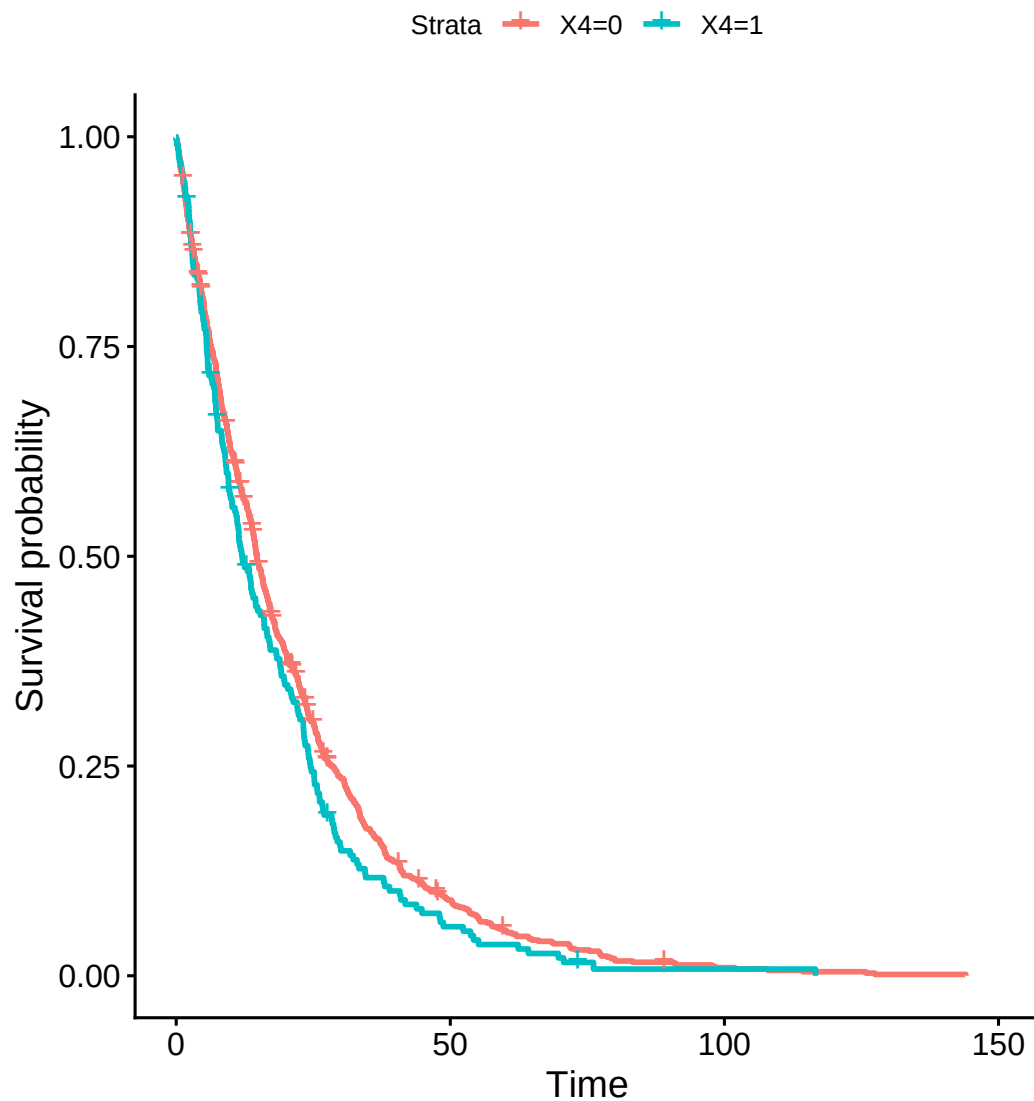
```
ggsurvplot(survfit(Surv(time, event)~X2, data), data)
```

```
ggsurvplot(survfit(Surv(time, event)~X3, data), data)
```



```
ggsurvplot(survfit(Surv(time, event)~X4, data), data)
```



```
# Fit the Aalen model
# All components of the model have nonparametric time-varying effects.
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value $H_0: B(t)=0$
(Intercept)	20.90	0.000
X1	6.25	0.000
X2	4.74	0.000
X3	2.21	0.313
X4	1.99	0.498

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value H_0 : constant effect
(Intercept)	0.908	0.000
X1	1.220	0.019
X2	0.719	0.093
X3	0.267	0.326
X4	0.157	0.272

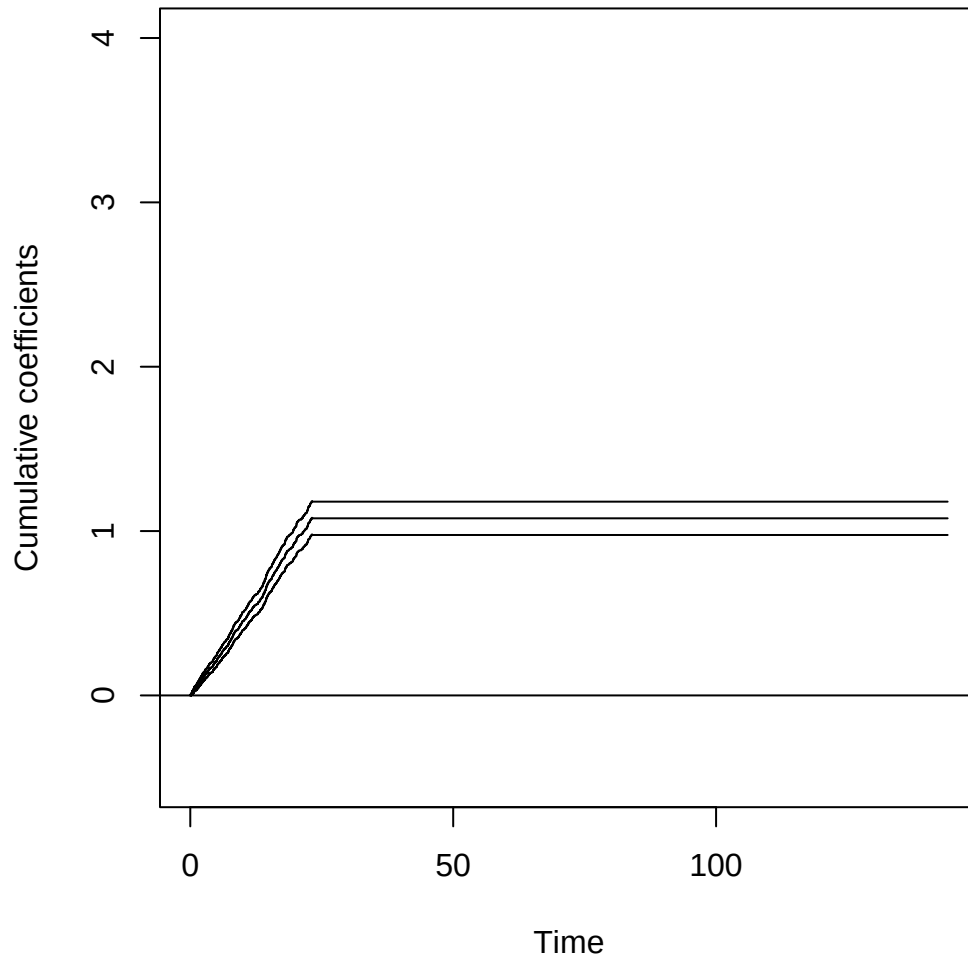
	Cramer von Mises test	p-value H_0 : constant effect
(Intercept)	38.300	0.000
X1	59.400	0.024
X2	15.900	0.130
X3	0.891	0.513
X4	0.357	0.416

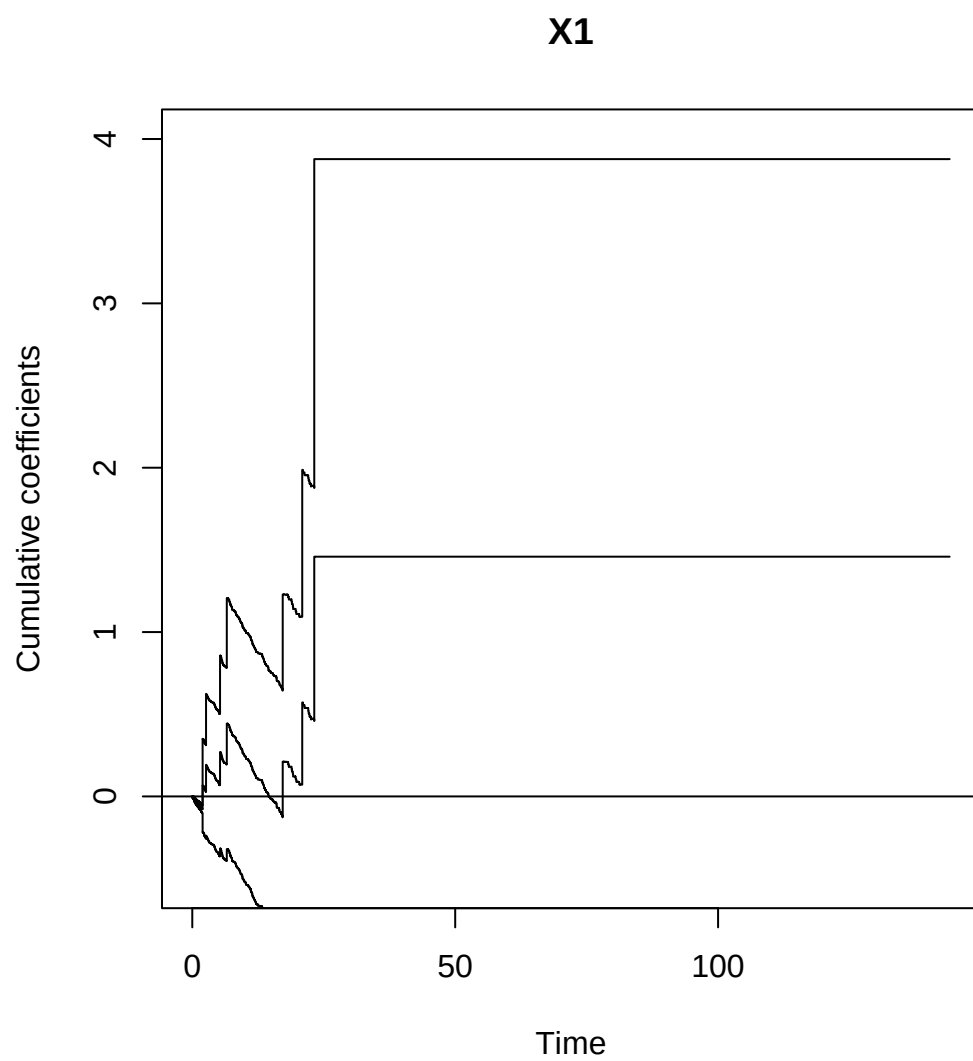
Call:

```
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,  
  data = data)
```

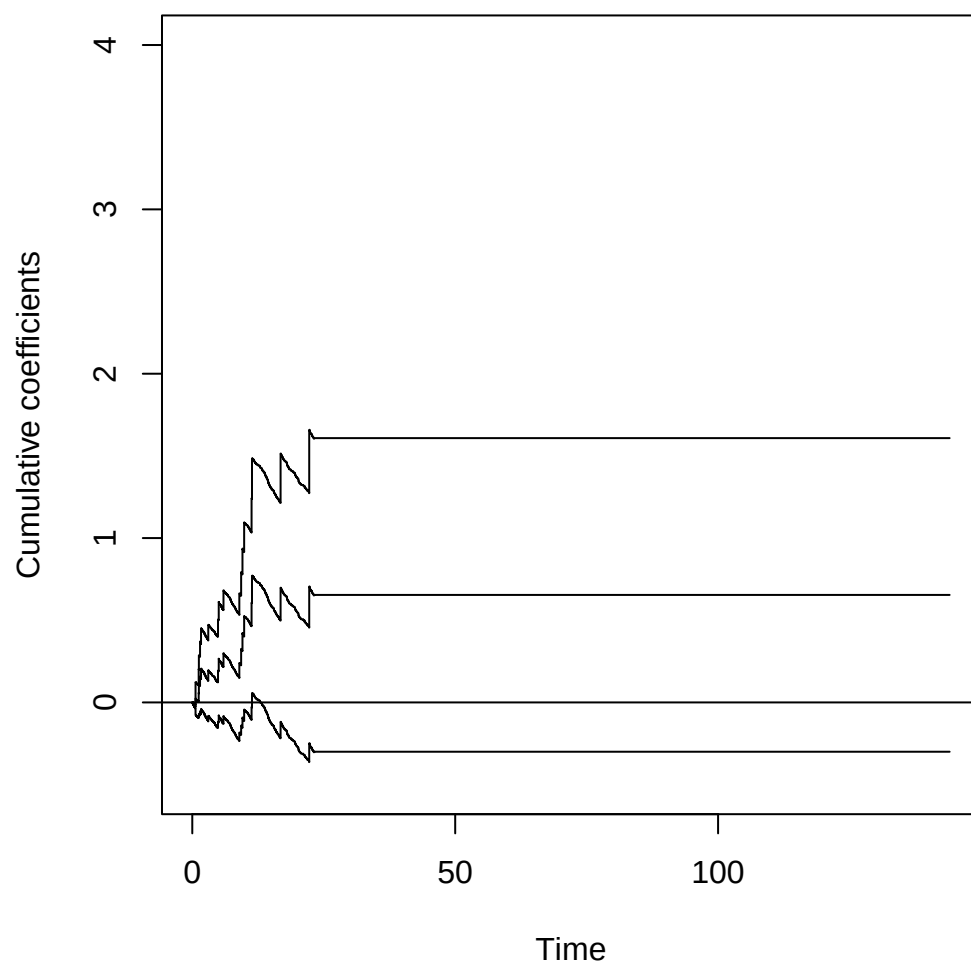
```
# Coefficients  
# cum <- aalen_fit$cum  
# tmax <- max(cum[, "time"])  
# B_at_tmax <- cum[nrow(cum), -c(1,2)]  
# est_beta <- as.vector(B_at_tmax / tmax)  
# est_beta  
  
# Plot the non-parametric cumulative regression coefficients with 95% CI  
# par(mfrow=c(1,2))  
plot(aalen_fit, ylim = c(-0.5,4))
```

(Intercept)

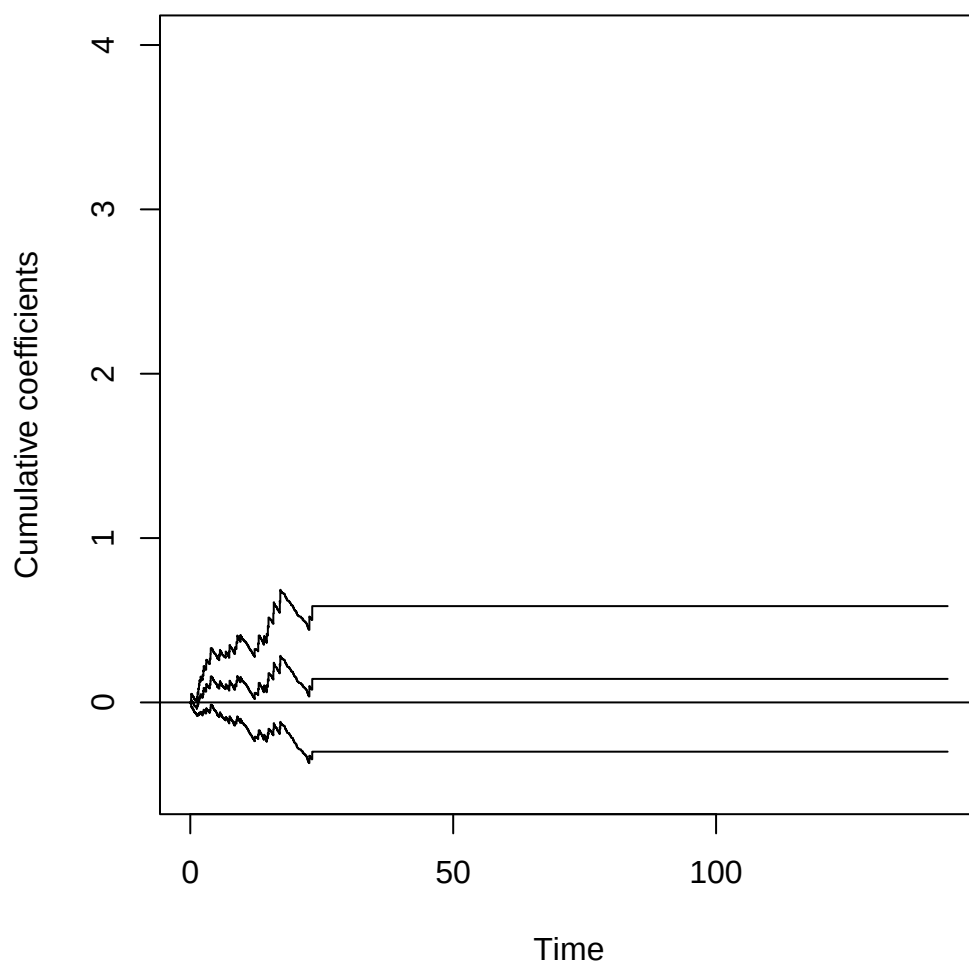


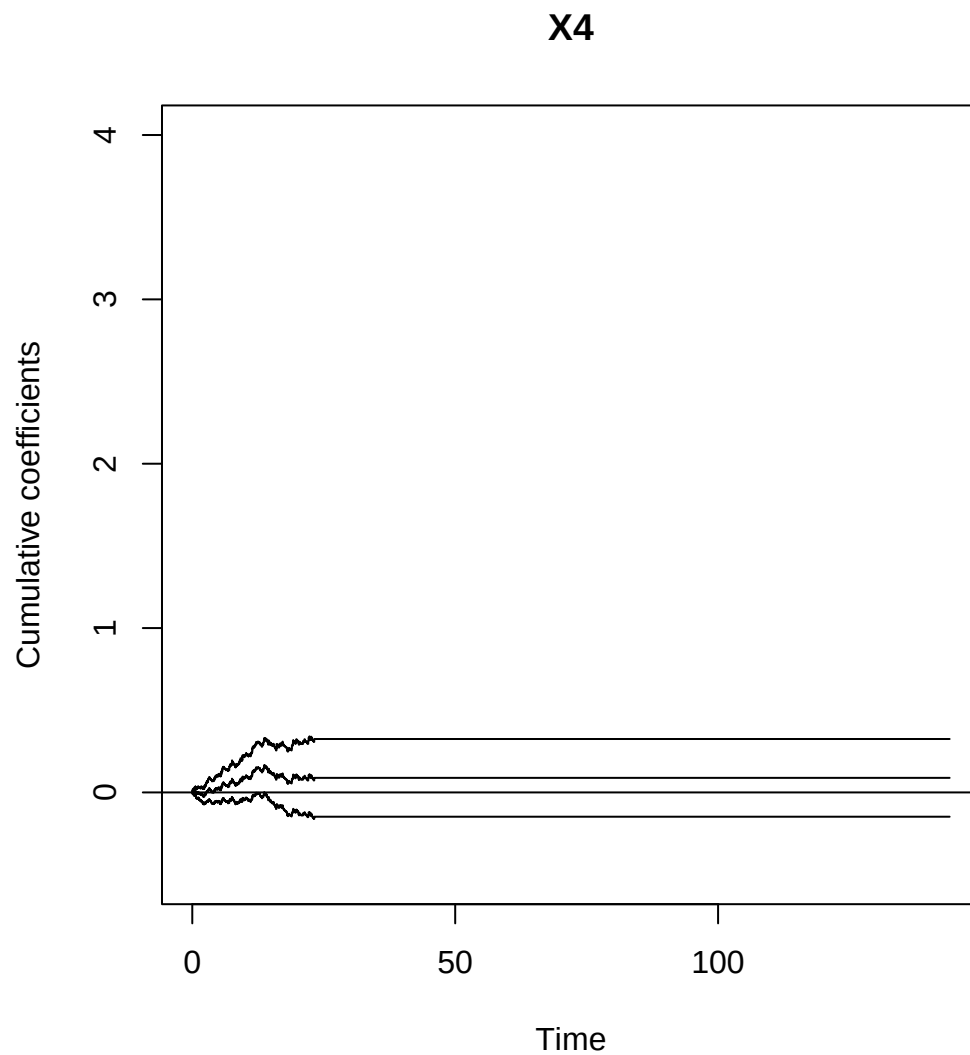


X2



X3





```
# Fit the model with the effect of X3 and X4 being constant
aalen_fit2 <- timereg::aalen(Surv(time, event)~
                             X1+X2+const(X3)+const(X4),
                             data = data)
summary(aalen_fit2)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	20.10		0
X1	7.89		0
X2	5.12		0

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ :constant effect
(Intercept)	0.831		0.000
X1	1.030		0.053
X2	0.722		0.072

	Cramer von Mises test	p-value	H ₀ :constant effect
(Intercept)	32.4		0.000
X1	42.4		0.058
X2	18.5		0.088

Parametric terms :

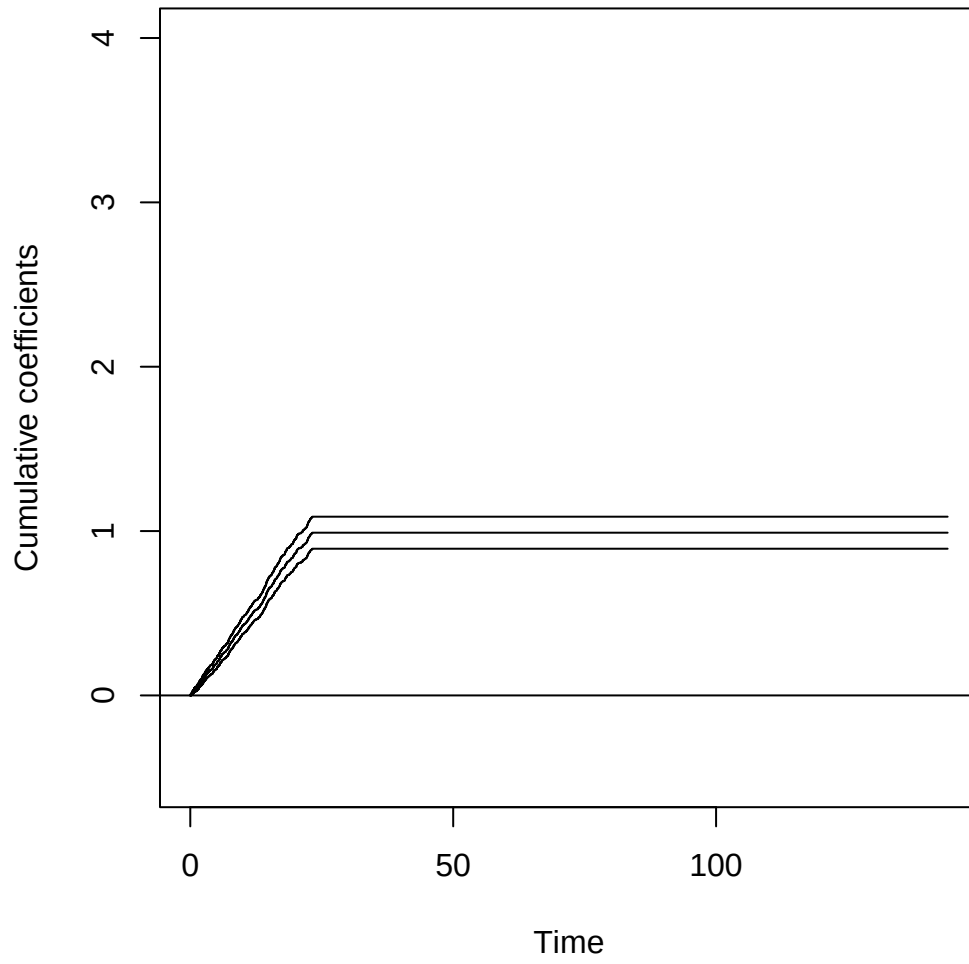
	Coef.	SE	Robust SE	z	P-val	lower2.5%	upper97.5%
const(X3)	0.0182	0.00765	0.00673	2.71	0.006760000000000	0.00321	0.0332
const(X4)	0.0217	0.00435	0.00315	6.89	0.000000000000563	0.01320	0.0302

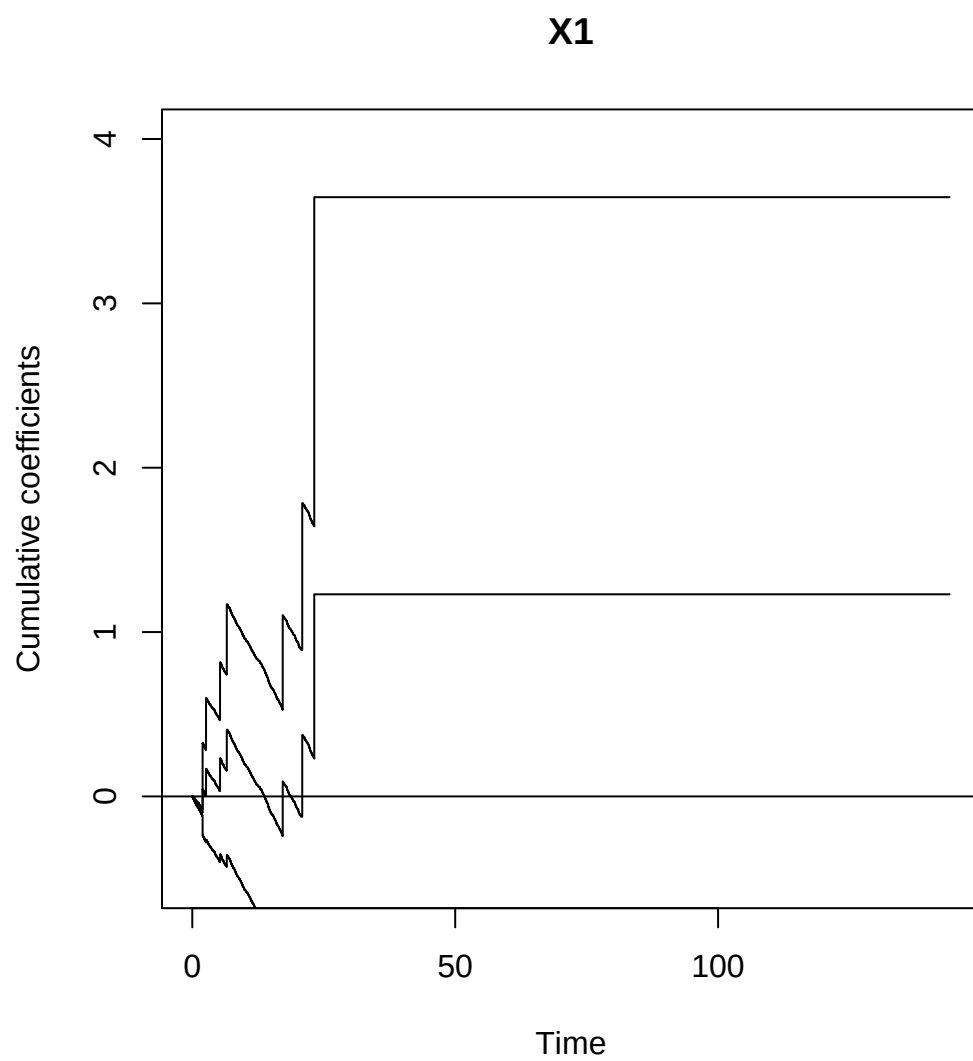
Call:

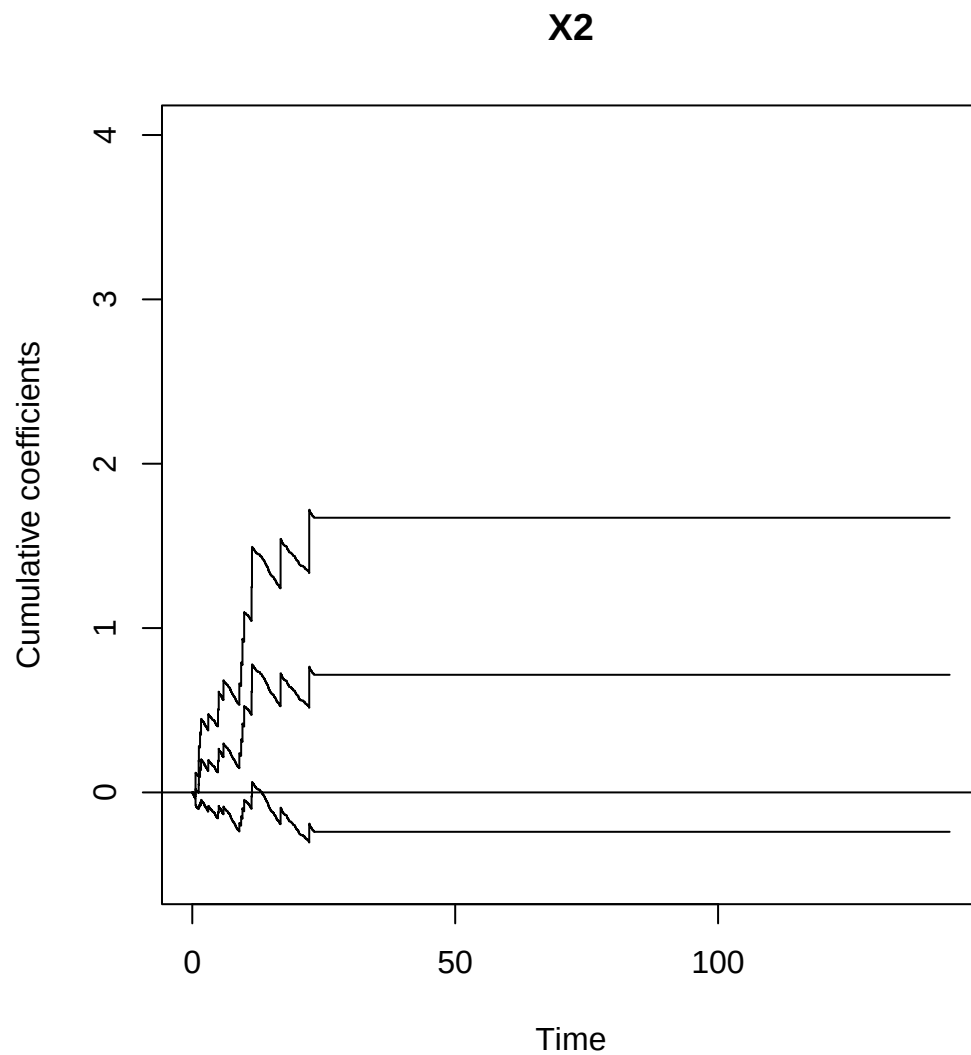
```
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + const(X3) +  
  const(X4), data = data)
```

```
# par(mfrow=c(1,2))  
plot(aalen_fit2, ylim = c(-0.5,4))
```

(Intercept)







```
# Fit the model with only time-constant effects
aalen_fit3 <- timereg::aalen(Surv(time, event)~
                             const(X1)+const(X2)+const(X3)+const(X4),
                             data = data)
summary(aalen_fit3)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	$H_0: B(t)=0$
(Intercept)	22.3		0

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H_0 :constant effect
(Intercept)	0.625		0.329
	Cramer von Mises test	p-value	H_0 :constant effect
(Intercept)	11		0.317

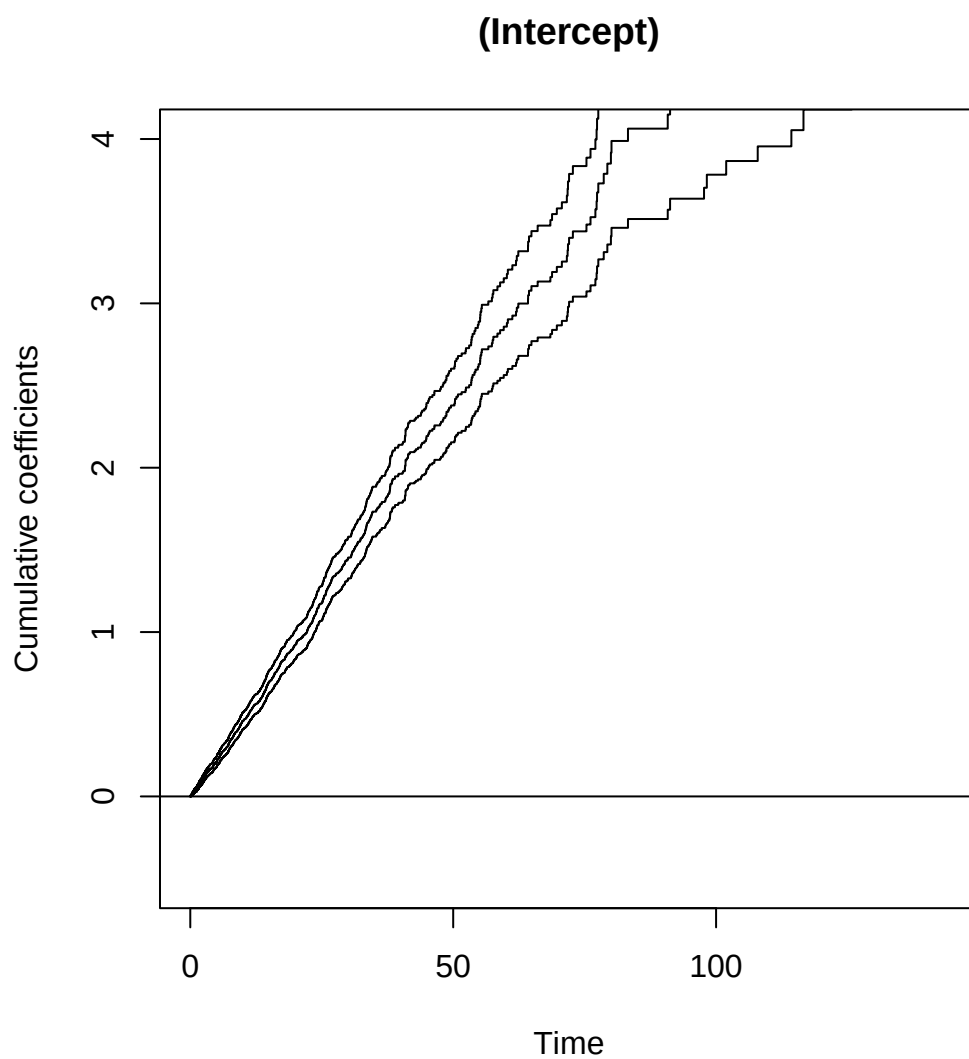
Parametric terms :

	Coef.	SE	Robust SE	z	P-val	lower2.5%	upper97.5%
const(X1)	0.03910	0.03420	0.02650	1.470	0.14100	-0.027900	0.1060
const(X2)	0.05170	0.02230	0.01880	2.750	0.00605	0.007990	0.0954
const(X3)	0.00349	0.00774	0.00823	0.424	0.67200	-0.011700	0.0187
const(X4)	0.00775	0.00440	0.00436	1.780	0.07570	-0.000874	0.0164

Call:

```
timereg::aalen(formula = Surv(time, event) ~ const(X1) + const(X2) +  
  const(X3) + const(X4), data = data)
```

```
plot(aalen_fit3, ylim = c(-0.5,4))
```



```
R <- 1000
test_aalen <- replicate(R, {
  data <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
  #'For constant effects: cumulative coefficient  $B_j(t)$   $\beta_j * t$ .
  #'Approximate  $\beta_j$  by  $B_j(t_{\max}) / t_{\max}$  using the last available
  #'cumulative estimate.
  #'timereg stores cumulative estimates in fit_obj$cum: matrix with columns
  #'(time, (coef, se) per covariate).
  fit_obj <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data)
  cum <- fit_obj$cum
```

```

tmax <- max(cum[, "time"])
B_at_tmax <- cum[nrow(cum), -c(1,2)]
est_beta <- as.vector(B_at_tmax / tmax)

data.frame(
  prev_X1 = mean(data$X1),
  prev_X2 = mean(data$X2),
  prev_X3 = mean(data$X3),
  prev_X4 = mean(data$X4),
  cens_rate = mean(1 - data$event),
  beta_X1 = est_beta[1],
  beta_X2 = est_beta[2],
  beta_X3 = est_beta[3],
  beta_X4 = est_beta[4]
), simplify = F)

colMeans(bind_rows(test_aalen))

```

prev_X1	prev_X2	prev_X3	prev_X4	cens_rate
0.00501900000	0.02010400000	0.04977500000	0.20003200000	0.04944400000
beta_X1	beta_X2	beta_X3	beta_X4	
0.00829913561	0.00311098494	0.00059683349	0.00001440651	

3.2 Cox-Aalen model

The covariates are generated *independently* as follows:

$$X = (X_1, X_2, X_3, X_4)$$

where

- $X_1 \sim \mathcal{B}(0.4)$
- $X_2 \sim \mathcal{B}(0.4)$
- $X_3 \sim \mathcal{B}(0.4)$
- $X_4 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1 and X_2 with coefficients $\beta_1 = 0.06$ and $\beta_3 = 0.005$, respectively. Multiplicative (proportional hazards) effects were assigned to X_3 and X_4 with coefficients $\beta_2 = 0.5$ and $\beta_4 = 0.2$, corresponding to hazard ratios of approximately 1.65 and 1.22.


```

# Set seed for reproducibility
set.seed(2026)

# Generate data
n <- 1000
prev_add <- c(0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 2, ncol = 2)
beta_add <- c(0.06, 0.005)
prev_mult <- 0.4
sigma_mult_bin <- diag(x = 1, nrow = 1, ncol = 1)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta_mult <- c(0.5, 0.2)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_cox_aalen(lambda0      = lambda0,
                                   beta_add      = beta_add,
                                   beta_mult     = beta_mult,
                                   prev_add      = prev_add,
                                   sigma_add     = sigma_add,
                                   prev_mult     = prev_mult,
                                   mean_mult     = mean_mult,
                                   sigma_mult_bin = sigma_mult_bin,
                                   sigma_mult_cont = sigma_mult_cont,
                                   target_cens   = target_cens,
                                   seed         = 2026)

c_max

```

```
[1] 261.5929
```

```

data <- simulate_cox_aalen_data(n, lambda0, beta_add, beta_mult,
                                prev_add, sigma_add,
                                prev_mult, mean_mult,
                                sigma_mult_bin, sigma_mult_cont,
                                c_max)

head(data)

```

	time	event	X1	X2	X3	X4
1	6.617109	1	1	1	0	-0.54856677
2	6.221355	1	0	1	0	-0.68120297
3	2.742523	1	1	1	1	0.33735955

```

4 3.837105      1 0 0 1 -0.29210344
5 3.260748      1 1 1 0 0.43125567
6 23.846538     1 0 0 0 -0.05632983

```

```

# Check prevalences / mean
colMeans(data %>% dplyr::select(starts_with("X")))

```

```

              X1                      X2
0.40000000000000002220446  0.399000000000000002131628
              X3                      X4
0.383000000000000000710543 -0.000000000000000001744308

```

```

# Check censoring rate
mean(1 - data$event)

```

```
[1] 0.043
```

```

# Kaplan-Meier
km_cox_aalen_fit <- survfit(Surv(time, event)~1, data)
km_cox_aalen_fit

```

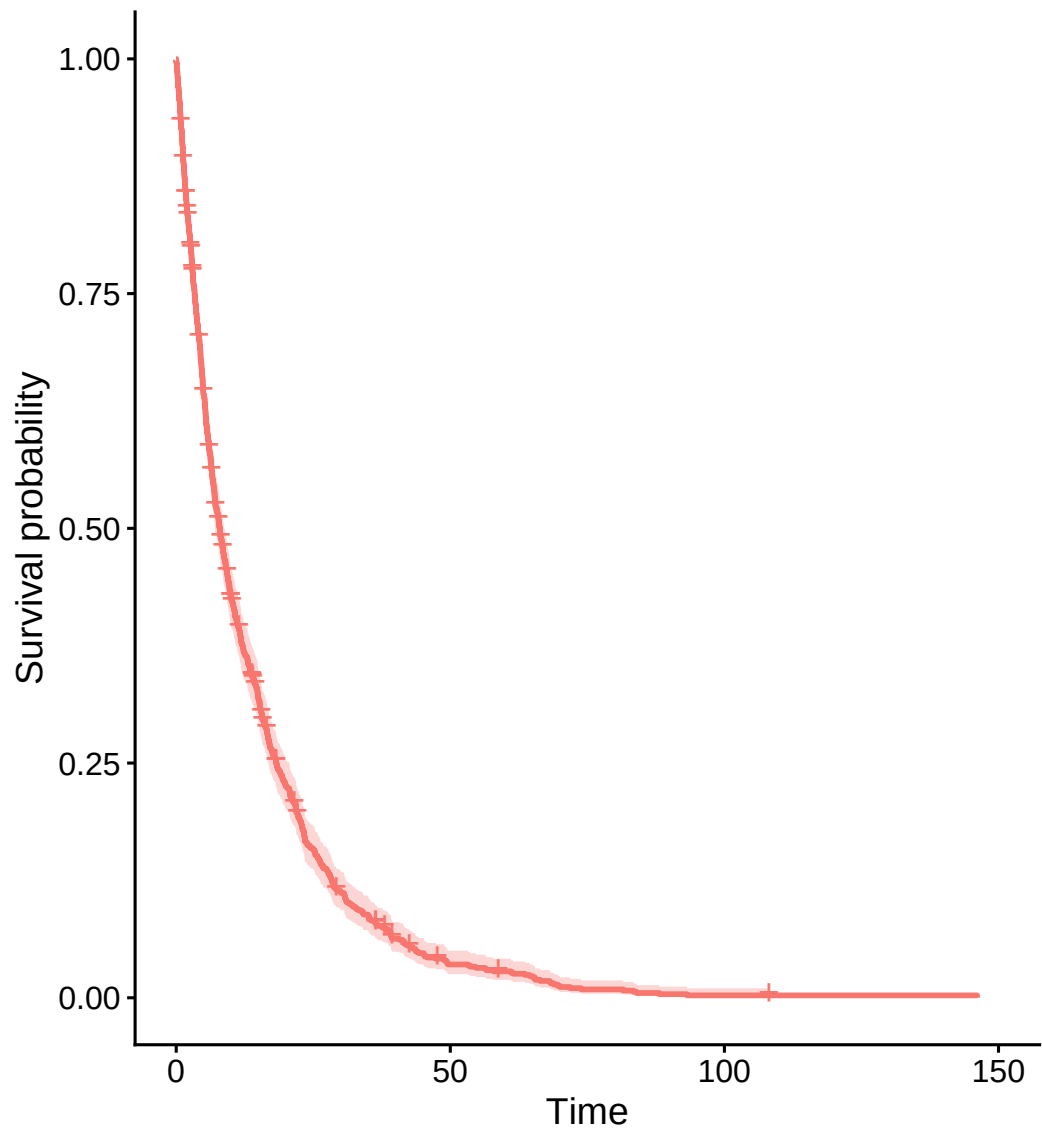
```
Call: survfit(formula = Surv(time, event) ~ 1, data = data)
```

```

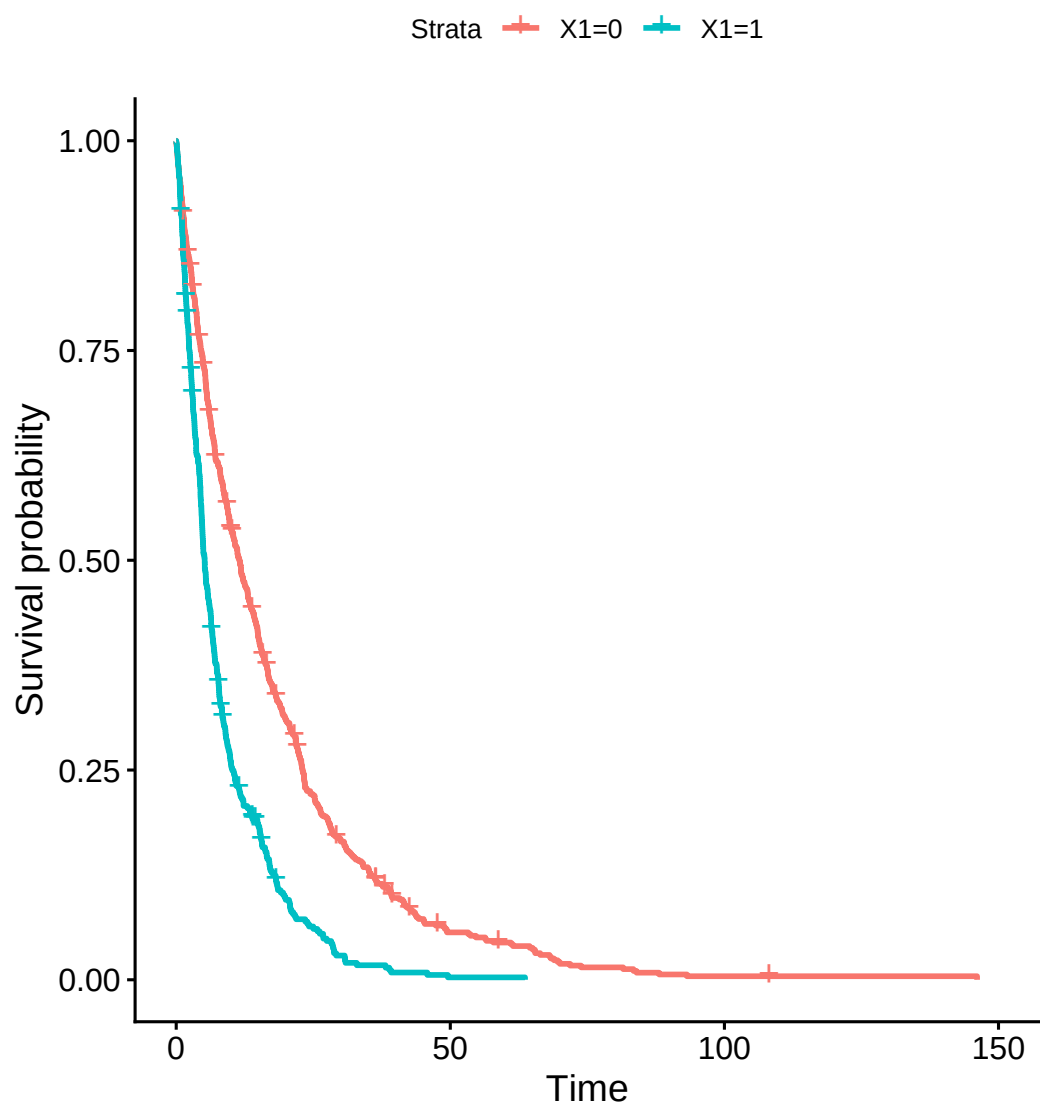
      n events median 0.95LCL 0.95UCL
[1,] 1000    957   7.98   7.05   8.81

```

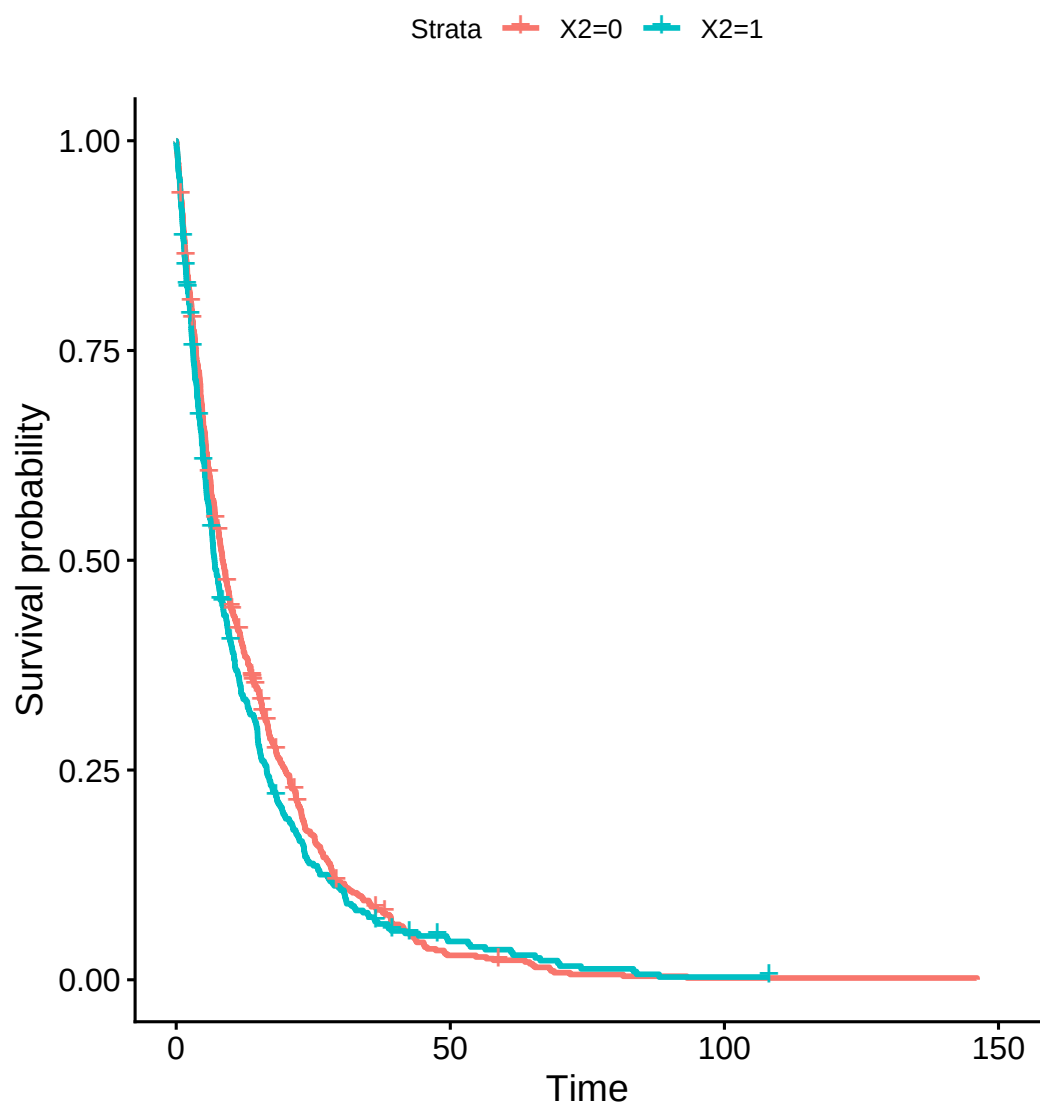
```
ggsurvplot(km_cox_aalen_fit, data = data, legend = "none")
```



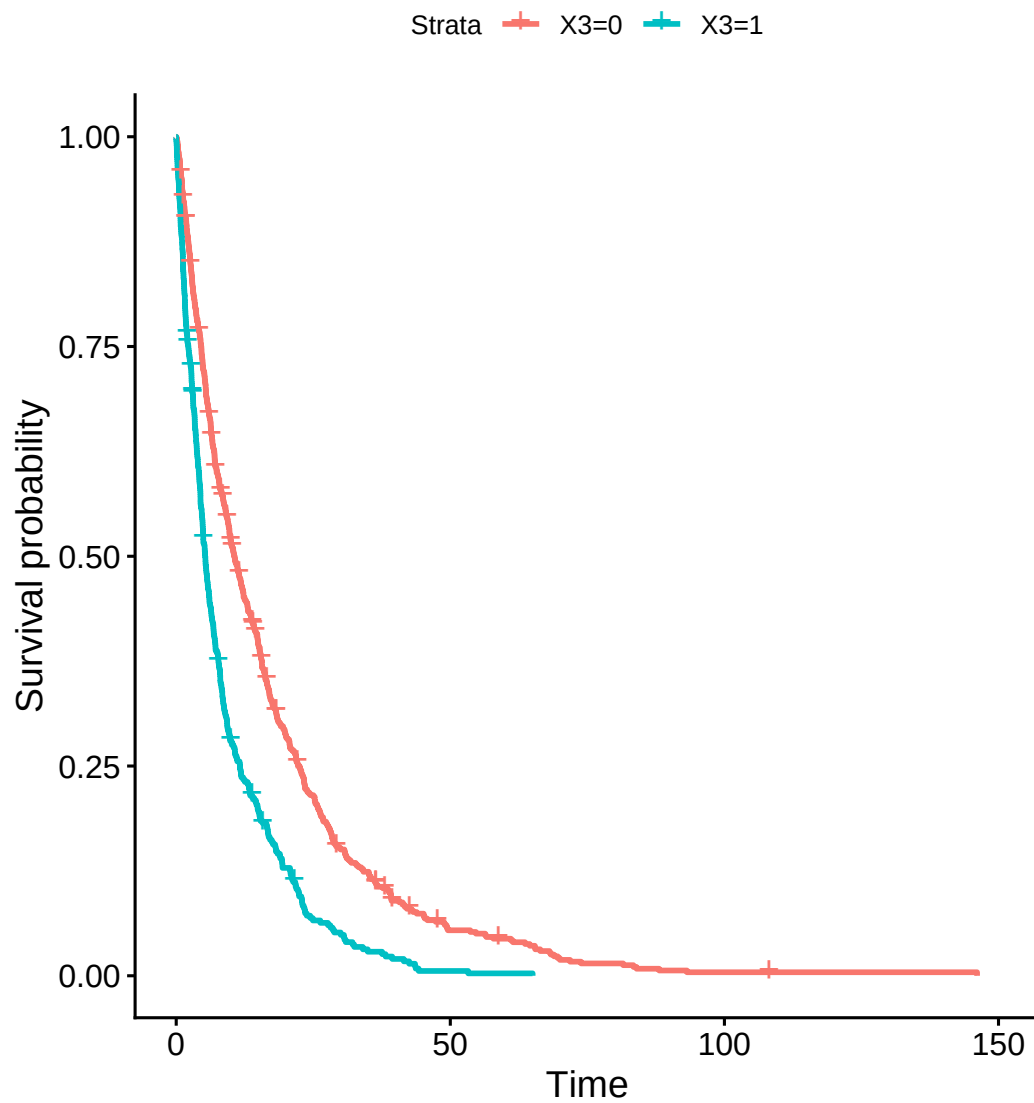
```
ggsurvplot(survfit(Surv(time, event)~X1, data), data)
```



```
ggsurvplot(survfit(Surv(time, event)~X2, data), data)
```



```
ggsurvplot(survfit(Surv(time, event)~X3, data), data)
```



```
# Fit Cox-Aalen model
cox_aalen_fit <- timereg::cox.aalen(
  Surv(time, event)~X1+X2+prop(X3)+prop(X4),
  data)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test not computed, sim=0

Proportional Cox terms :

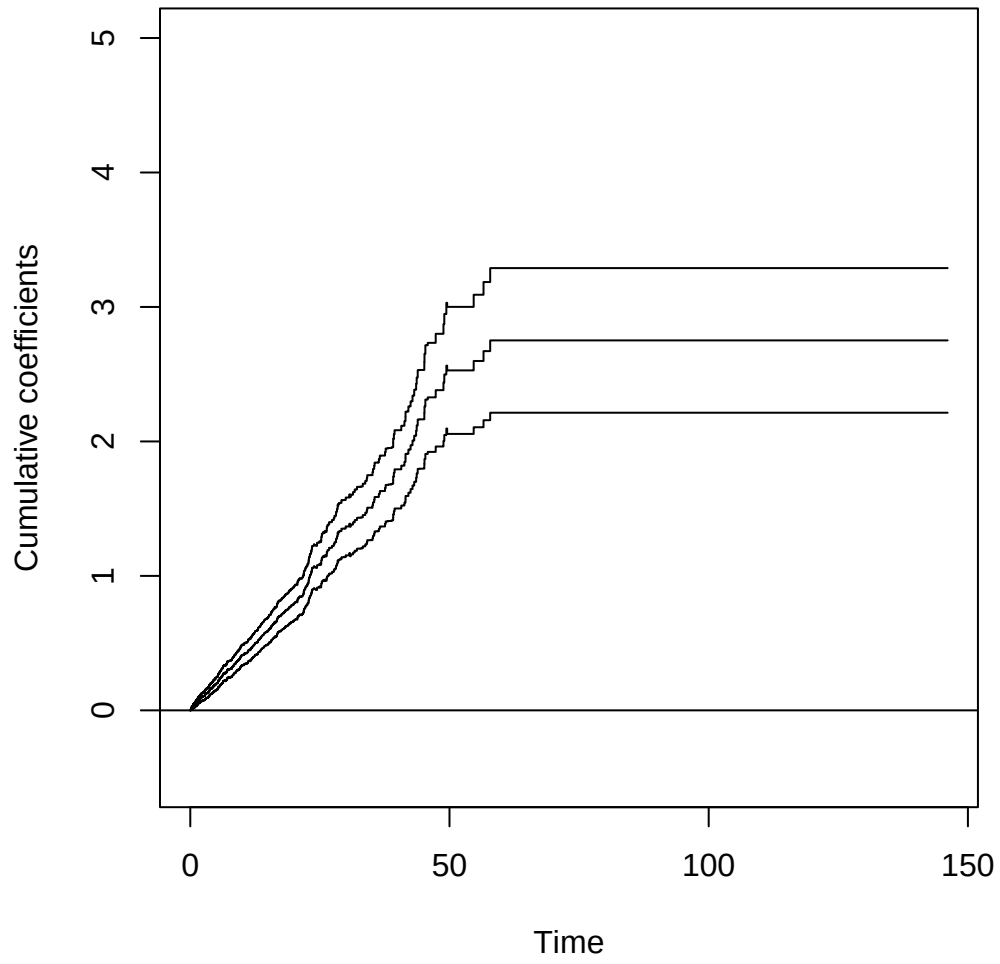
	Coef.	SE	Robust SE	D2log(L)^-1	z	P-val	lower2.5%
prop(X3)	0.673	0.0691	0.0672	0.0692	10.00	0.000000000000	0.538
prop(X4)	0.197	0.0332	0.0320	0.0331	6.17	0.000000000701	0.132
	upper97.5%						
prop(X3)	0.808						
prop(X4)	0.262						

Test of Proportionality

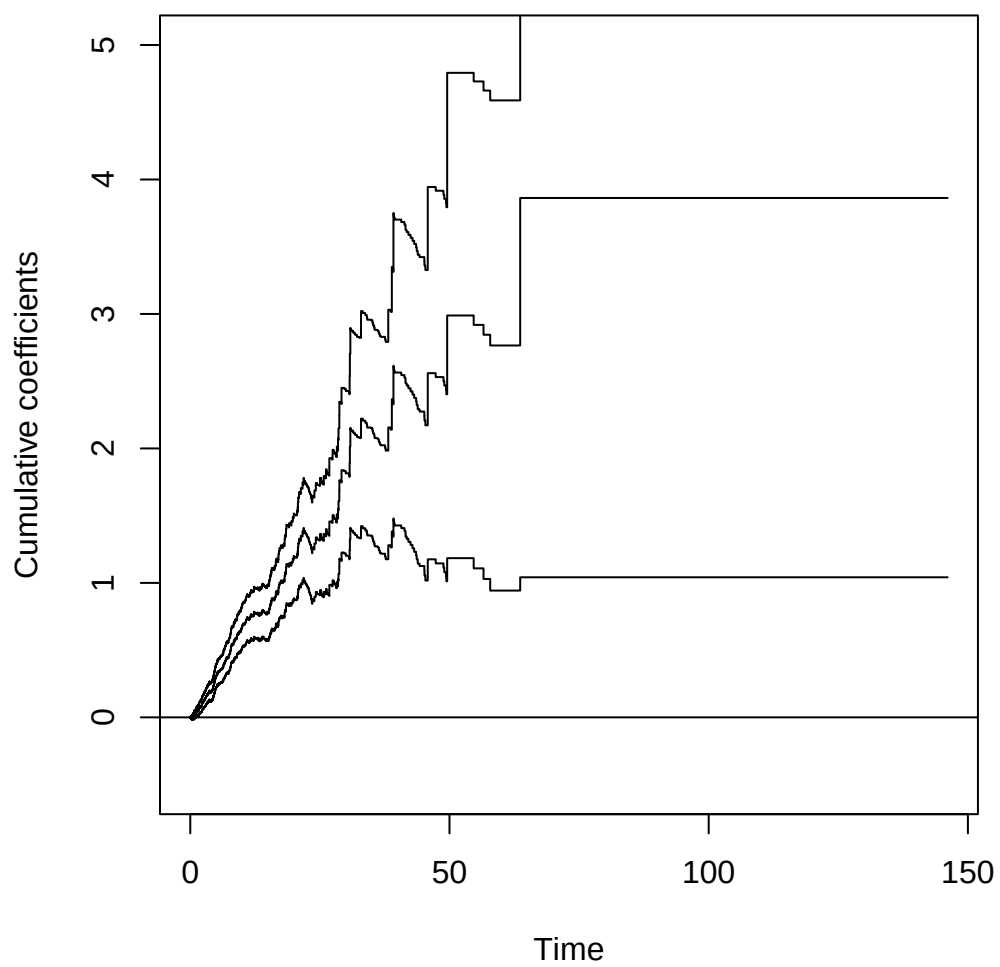
	sup hat U(t)	p-value H_0
prop(X3)	12.8	0.316
prop(X4)	24.2	0.410

```
# Plot additive Aalen terms
# par(mfrow = c(1,2))
plot(cox_aalen_fit, ylim = c(-0.5,5))
```

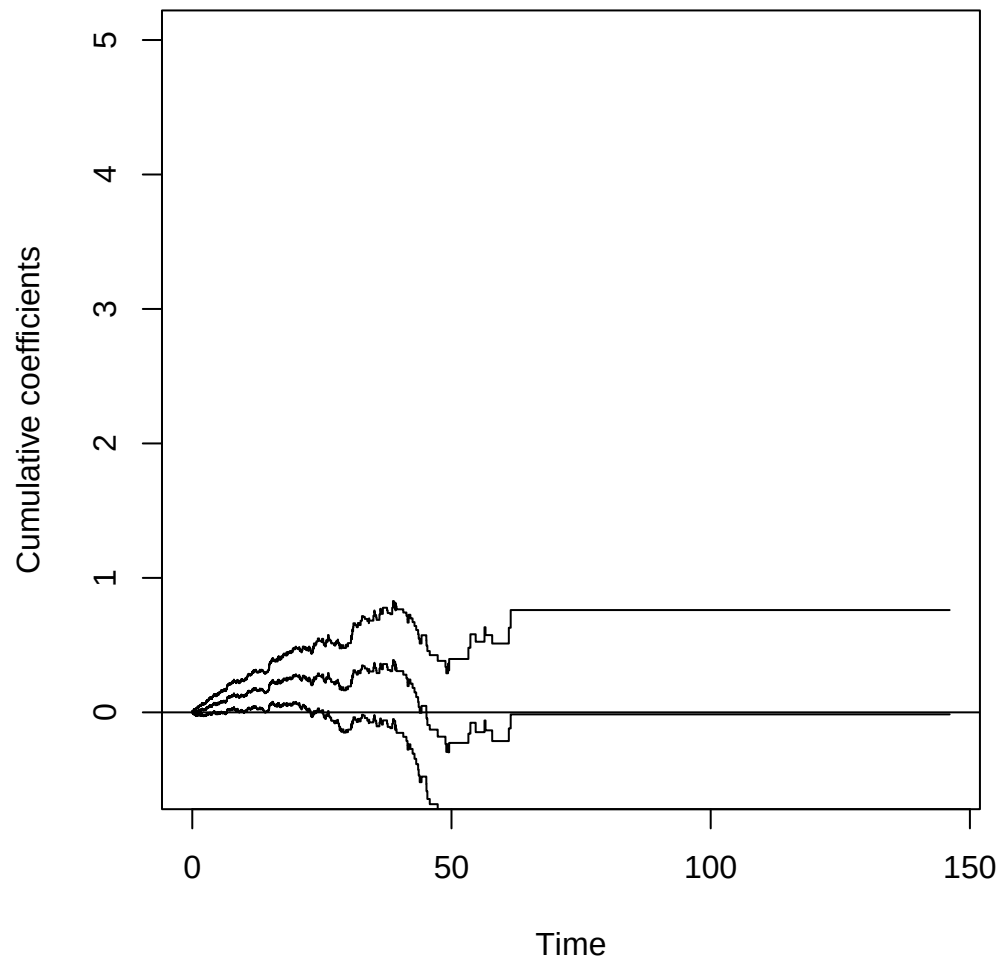
(Intercept)



X1



X2



```
# Allow X2 to be time-constant
cox_aalen_fit2 <- timereg::cox.aalen(
  Surv(time, event)~X1+const(X2)+prop(X3)+prop(X4),
  data, robust = TRUE)
summary(cox_aalen_fit2)
```

Cox-Aalen Model

Test for Aalen terms

Test not computed, sim=0

Proportional Cox terms :

	Coef.	SE Robust	SE	D2log(L)^-1	z	P-val	lower2.5%
prop(X3)	0.673	0.0691	0.0672	0.0692	10.00	0.000000000000	0.538
prop(X4)	0.197	0.0332	0.0320	0.0331	6.17	0.000000000701	0.132
upper97.5%							
prop(X3)	0.808						
prop(X4)	0.262						

Test of Proportionality

	sup hat U(t)	p-value H_0
prop(X3)	12.8	0.328
prop(X4)	24.2	0.378

```
# Proportional excess terms of model  
cox(cox_aalen_fit2)
```

Cox-Aalen Model

Test for Aalen terms

Test not computed, sim=0

Proportional Cox terms :

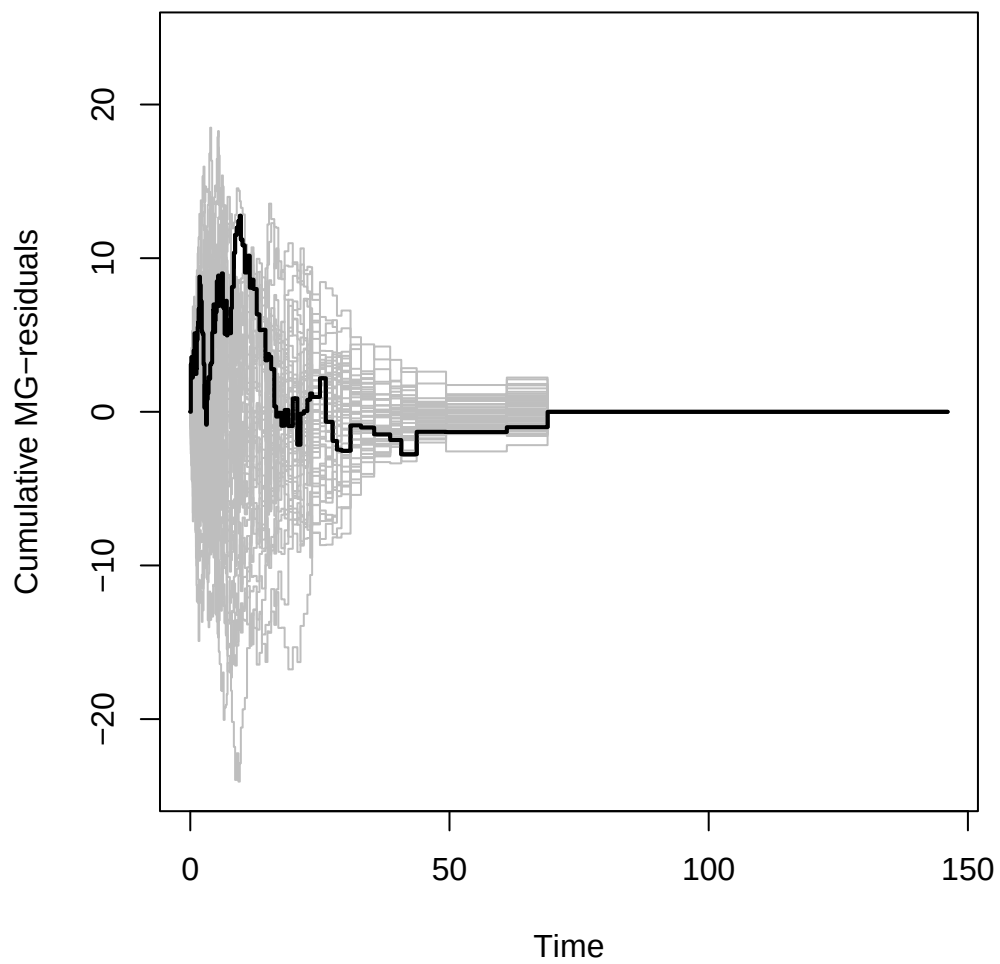
	Coef.	SE Robust	SE	D2log(L)^-1	z	P-val	lower2.5%
prop(X3)	0.673	0.0691	0.0672	0.0692	10.00	0.000000000000	0.538
prop(X4)	0.197	0.0332	0.0320	0.0331	6.17	0.000000000701	0.132
upper97.5%							
prop(X3)	0.808						
prop(X4)	0.262						

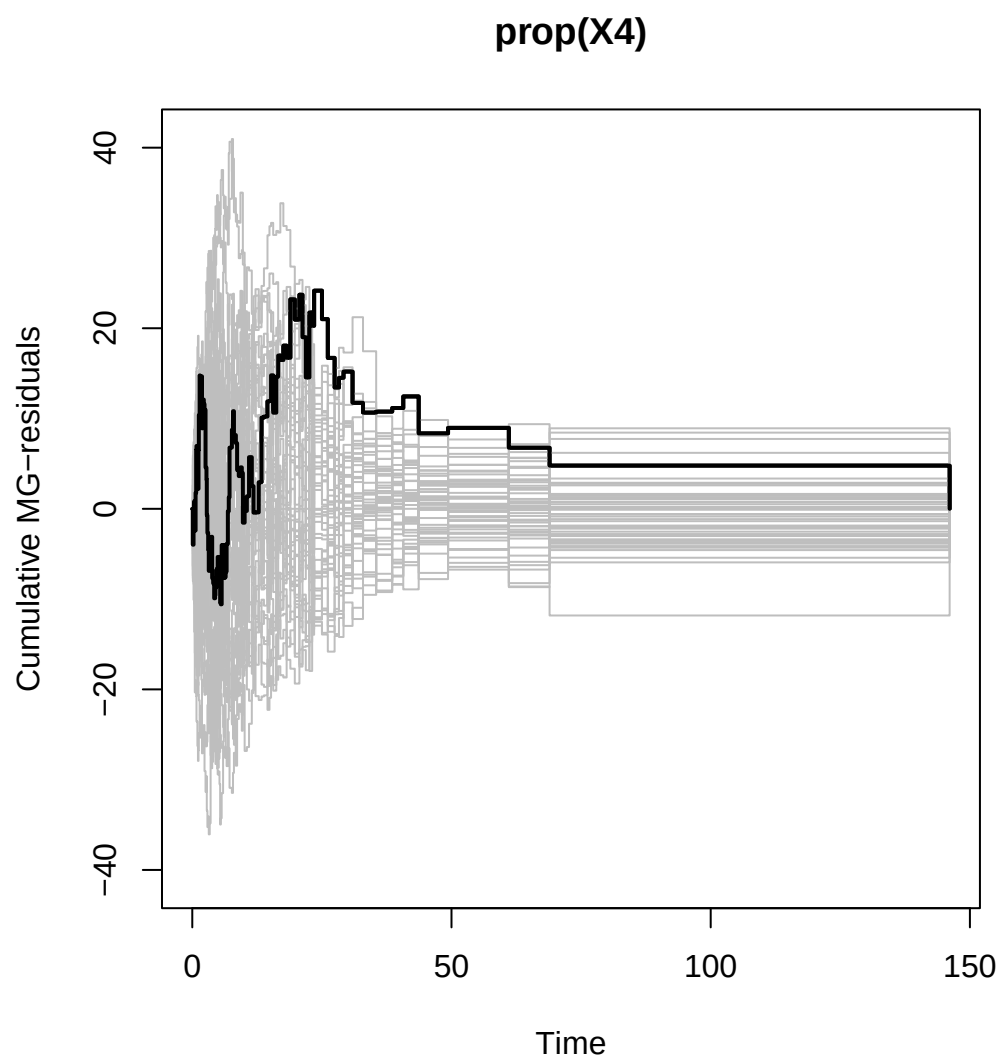
Test of Proportionality

	sup hat U(t)	p-value H_0
prop(X3)	12.8	0.328
prop(X4)	24.2	0.378

```
# par(mfrow=c(1,2))  
plot(cox_aalen_fit2,score= TRUE)
```

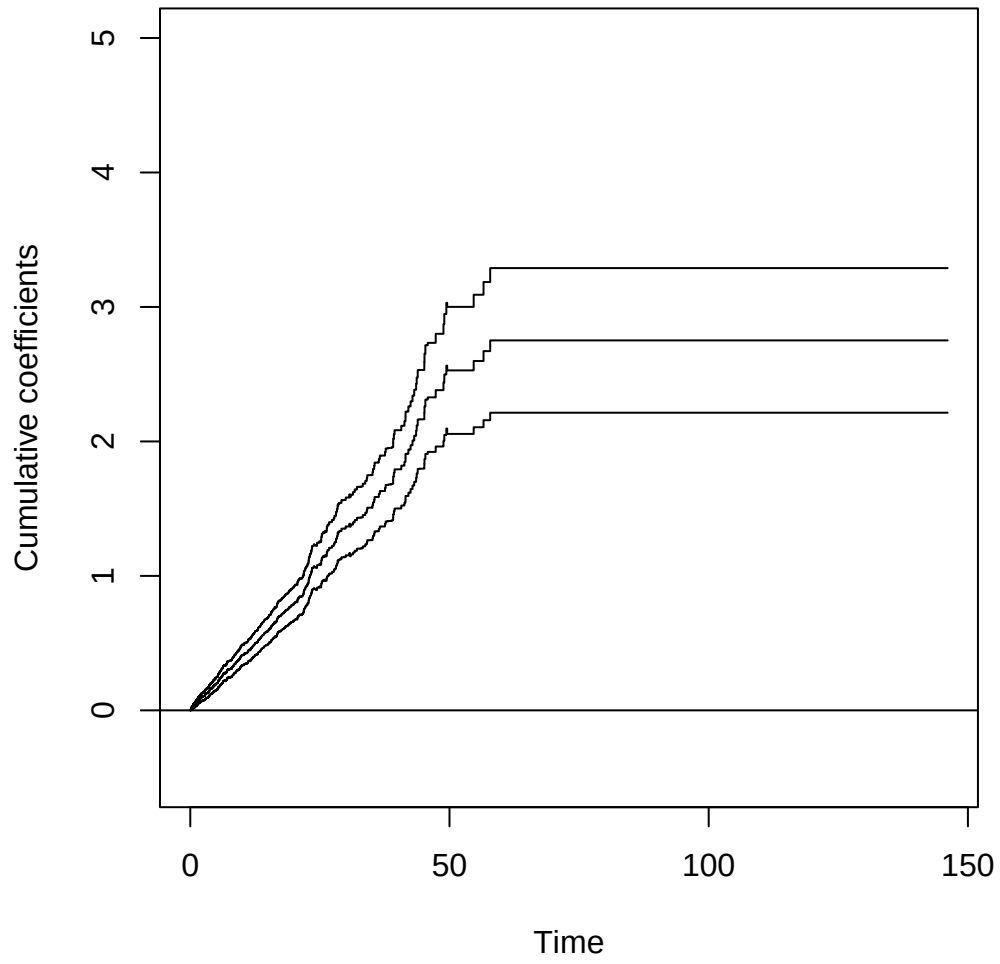
prop(X3)



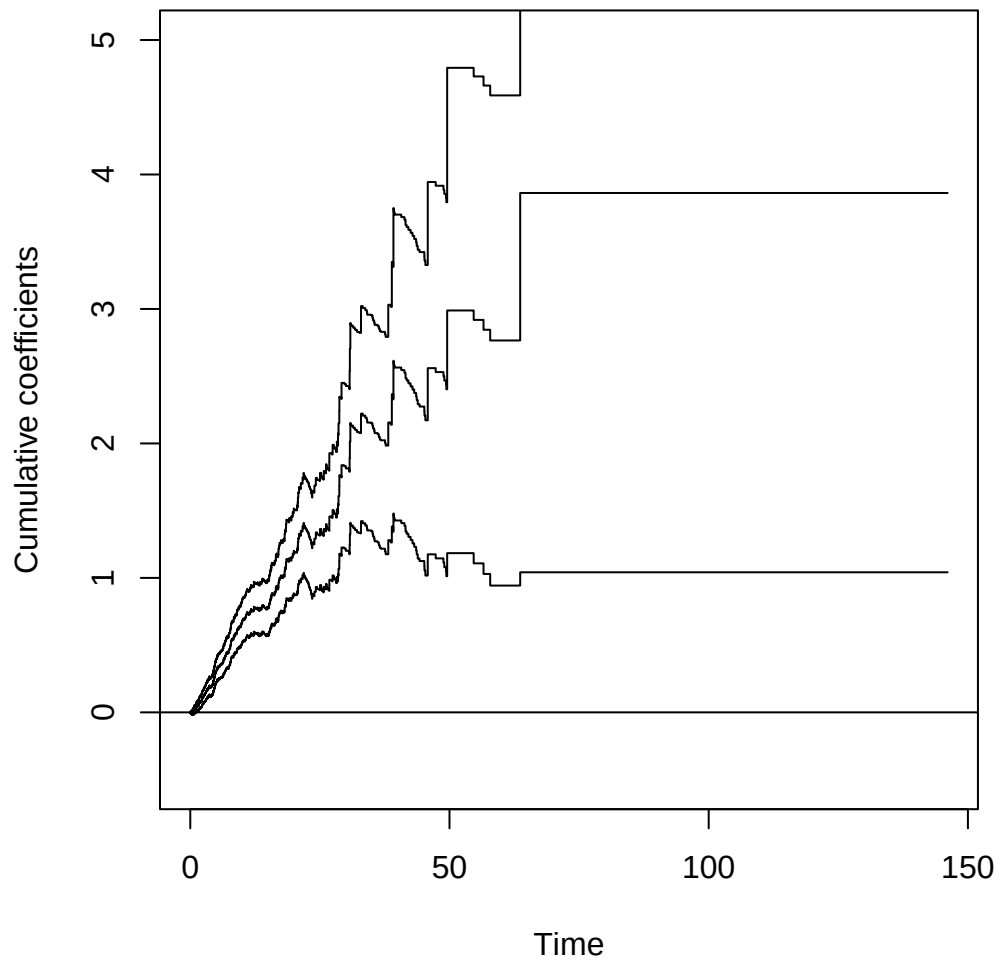


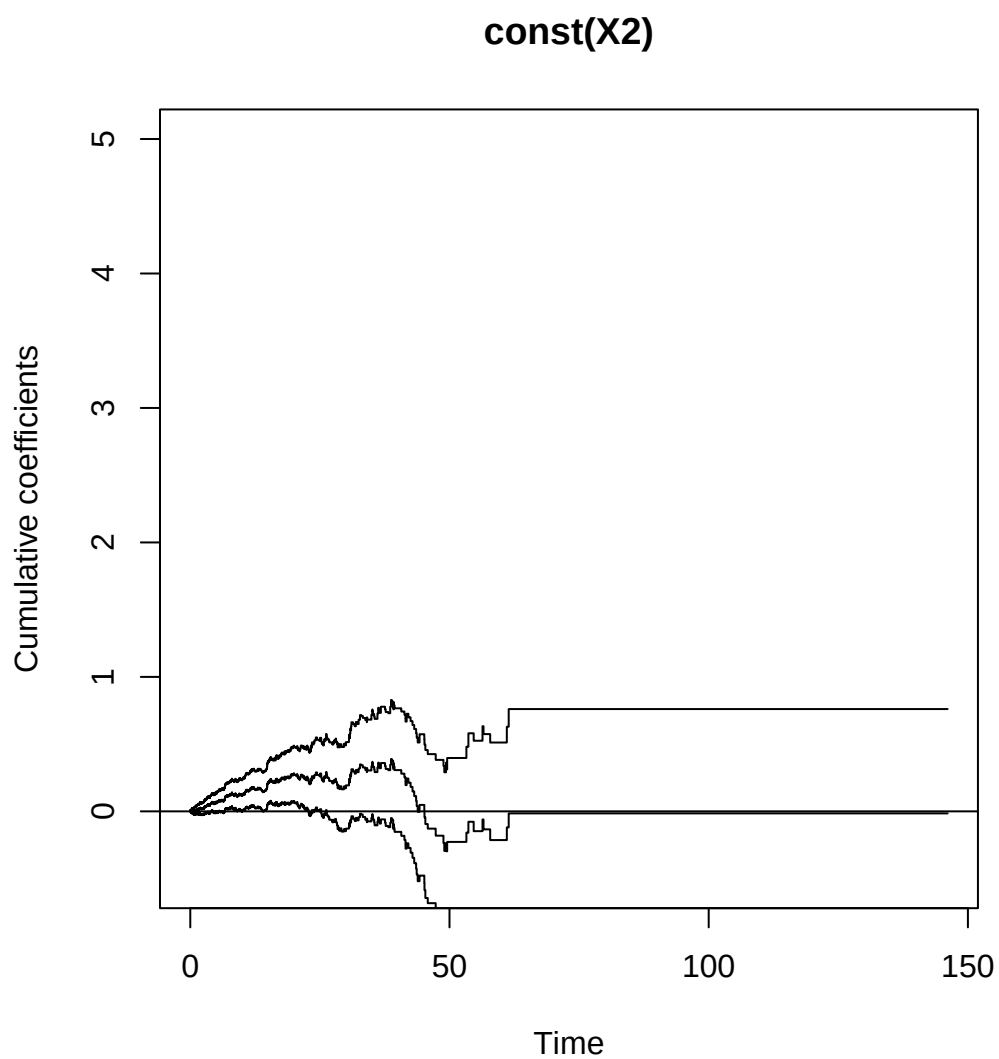
```
# Plot additive Aalen terms  
# par(mfrow = c(1,2))  
plot(cox_aalen_fit2, ylim = c(-0.5, 5))
```

(Intercept)



X1





```
R <- 1000
test_cox_aalen <- replicate(R, {
  data <- simulate_cox_aalen_data(n, lambda0, beta_add, beta_mult,
    prev_add, sigma_add,
    prev_mult, mean_mult,
    sigma_mult_bin, sigma_mult_cont,
    c_max)

  cox_aalen_fit <- timereg::cox.aalen(
    Surv(time, event)~X1+X2+prop(X3)+prop(X4),
    data = data)
```



```

coef <- data.frame(coef(cox_aalen_fit))
data.frame(
  prev_X1 = mean(data$X1),
  prev_X2 = mean(data$X2),
  prev_X3 = mean(data$X3),
  mean_X4 = mean(data$X4),
  cens_rate = mean(1 - data$event),
  beta_X3 = coef[1, "Coef."],
  beta_X4 = coef[2, "Coef."]
)}, simplify = F)

colMeans(bind_rows(test_cox_aalen))

```

	prev_X1	prev_X2
	0.400019999999999866950873	0.3999559999999999782005489
	prev_X3	mean_X4
	0.3999099999999999877076107	-0.00000000000000000003731662
	cens_rate	beta_X3
	0.04959100000000000031339376	0.51453300000000000181259452
	beta_X4	
	0.189514399999999995242916	